
THE EVOLUTION OF MACHINE LEARNING AND AI

Week 3 Full Booklet

Optional reference · the required reading is the 18-page Core Booklet

How Machines Learn, Fast: Classical Models, Neural Networks, and the Loop Underneath Them

Award	Certificate in Artificial Intelligence and Machine Learning Engineering (30 ECTS)
Parent programme	Higher Diploma in Science in Computer Science (NFQ Level 8)
Module	The Evolution of Machine Learning and AI (5 ECTS, Semester 1)
Week	Week 3 of 12 · Sprint 1: Fast foundations and responsible AI up front
Indicative content	Items 2, 3 and 7, classical ML and early neural networks; training, backpropagation and GPU acceleration; deep learning frameworks (PyTorch)
Learning outcome	LO2: Characterise the training of the main families of model · LO4: Identify the tools and frameworks used in contemporary AI and ML
Assessed in	Mid-module quiz, Week 6 (50% of module mark)
Companions	Gradient Descent Explorer · Week 3 live lecture · Lab: seven classical models, backpropagation from scratch, then PyTorch

Read this first. This is the **full** account of Week 3, kept for students who want to go deeper. **You are not required to read it.** The required reading is the short **Core Booklet**, and the Week 6 quiz can only ask what the Core Booklet's red boxes say. The **Reading Map** on the next page marks every chapter here as CORE (also covered, more briefly, in the Core Booklet) or EXTRA (optional). If you are short of time, read the Core Booklet and stop. Boxes marked ◯ **Deeper Detail** go beyond what the lecture had time for. Boxes marked **Everyday Analogy** are the mental pictures the lecture used: they are the fastest route to remembering a concept under exam conditions. Boxes marked **In Plain English** restate an idea with every piece of jargon removed; if a section loses you, skip to that box and come back. Boxes marked **Why This Matters** say what goes wrong if you do not understand the idea, and which later week depends on it. Boxes marked  **Worked Example** are small enough to check with a pencil.

Contents

1	What Learning Actually Means	6
1.1	The one-sentence version	6
1.2	The three decisions	6
1.3	Parameters and hyperparameters	7
1.4	Supervised, unsupervised, and everything else	8
2	Reading the Mathematics	9
2.1	The symbols, one at a time	9
2.2	Reading a formula out loud	10
3	The Training Loop	12
3.1	Where the difficulty actually is	12
3.2	The hillwalker	13
4	Linear Regression	14
4.1	The model	14
4.2	The loss	14
4.3	Fitting it	15
4.4	Why fit it first, always	15
5	Logistic Regression	17
5.1	The sigmoid	17
5.2	The loss	17
5.3	It is still a linear classifier	18
6	Decision Trees	19
6.1	The idea	19
6.2	How the machine chooses the questions	19
6.3	What trees buy and what they cost	20
6.4	The memorisation problem	20
7	Random Forests	21
7.1	The idea	21
7.2	Why averaging works	21
7.3	The price	22
8	Support Vector Machines	23
8.1	The better question	23
8.2	Soft margins	23
8.3	The kernel trick	24
9	Capacity: Underfitting and Overfitting	26
9.1	The two ways to fail	26
9.2	Watching it happen	26
9.3	The bias-variance decomposition	27
9.4	What to do about it	28
10	The Perceptron and Its Wall	30
10.1	One unit	30
10.2	The learning rule	30
10.3	The theorem, and its condition	31
10.4	XOR	31
11	Gradient Descent	33
11.1	Why the derivative is exactly the right object	33
11.2	Batch, stochastic, mini-batch	34
12	The Learning Rate	36

12.1	The four failure modes	36
12.2	Two observations that matter more than the table	36
13	Local Minima, Saddles, and Momentum	38
13.1	The problem, demonstrated	38
13.2	Why deep networks suffer less than you would expect	38
13.3	Momentum	39
14	Backpropagation	41
14.1	The problem	41
14.2	The solution: the chain rule	41
14.3	The network, named	42
14.4	The two passes	42
14.5	The code	43
14.6	Check your gradient	43
14.7	And then XOR falls over	44
15	Activations and the Vanishing Gradient	46
15.1	Why any non-linearity at all	46
15.2	The three you need	46
15.3	The vanishing gradient	47
16	PyTorch, and What It Is Doing for You	49
16.1	Three things a framework gives you	49
16.2	The five lines	49
16.3	Line for line against what you wrote	49
16.4	The trap	50
17	Why It Waited Until 2012	52
18	A Field Guide: Seven Questions for Any Model You Train	53
19	The Revision Core: The Whole Week on Two Pages	54
19.1	The loop	54
19.2	The three decisions	54
19.3	The four model families	54
19.4	Capacity	54
19.5	XOR and the hidden layer	54
19.6	Gradient descent	54
19.7	Backpropagation	55
19.8	Ten one-line lessons	55
20	Glossary in Plain English	56
21	Question Bank: Practising for Week 6	58
21.1	Short answer: two to three sentences each	58
21.2	Applied: read the numbers	59
21.3	Discussion prompts	59
22	Further Reading	60
	Appendix: How Week 3 Maps to the Module	61

Reading map: what is core and what is extra

Every chapter below is labelled. **CORE** means the idea is examinable, and the Core Booklet already covers it in plain English; read the chapter here only if you want the fuller version. **EXTRA** means optional: interesting, not examined, safe to skip. Times are for a careful single read.

Ch.	Title	Status	Time
1	What Learning Actually Means	CORE	10 min
2	Reading the Mathematics	EXTRA	15 min
3	The Training Loop	CORE	8 min
4	Linear Regression	CORE	10 min
5	Logistic Regression	CORE	8 min
6	Decision Trees	CORE	10 min
7	Random Forests	EXTRA	6 min
8	Support Vector Machines	CORE	12 min
9	Capacity: Underfitting and Overfitting	CORE	15 min
10	The Perceptron and Its Wall	CORE	12 min
11	Gradient Descent	CORE	12 min
12	The Learning Rate	CORE	10 min
13	Local Minima, Saddles, and Momentum	EXTRA	12 min
14	Backpropagation	CORE	20 min
15	Activations and the Vanishing Gradient	EXTRA	12 min
16	PyTorch, and What It Is Doing for You	CORE	12 min
17	Why It Waited Until 2012	CORE	6 min
18	A Field Guide: Seven Questions	EXTRA	4 min
19	The Revision Core: The Whole Week on Two Pages	CORE	8 min
20	Glossary in Plain English	CORE	6 min
21	Question Bank	EXTRA	30 min
22	Further Reading	EXTRA	2 min

Inside a CORE chapter, the boxes marked  Deeper Detail and  Worked Example are themselves extra. The examinable statements are exactly the red ★ boxes in the Core Booklet; nothing here adds to that list.

1 What Learning Actually Means

COVERS	The three decisions behind every learning system, and the vocabulary for the rest of the module
TIME	Read first, about 20 minutes
IN BRIEF	“Machine learning” is fitting a function to data by minimising a loss. Everything else in this module is a different answer to one of three questions.

Week 1 asked what happened to this field. Week 2 asked who answers when it goes wrong. This week asks the mechanical question underneath both: **what does a machine actually do when it learns?**

The honest answer is far less mysterious than the vocabulary suggests, and it is worth stating plainly before anything complicates it.

1.1 The one-sentence version

A machine learning system is handed a pile of examples and asked to find a **rule** that turns inputs into outputs. It does this by guessing a rule, scoring how badly the guess does, adjusting the guess slightly, and repeating.

That is the whole thing. Everything technical in the next fourteen chapters is either a way of describing what kinds of rule are allowed, a way of scoring a bad guess, or a way of adjusting the guess intelligently.

IN PLAIN ENGLISH

Imagine you are trying to work out the rule a shop uses to price second-hand books, and nobody will tell you. All you have is a list of books and what each one sold for.

You guess: “**maybe it’s €2 per hundred pages.**” You check that guess against the list and find you are wrong by about €4 a book on average. You adjust: “**maybe €2 per hundred pages, plus €3 if it’s a hardback.**” You check again; now you are wrong by €2.50. You keep going.

That is machine learning. The only real differences are that the computer adjusts millions of numbers instead of two, and it does the adjusting a few million times rather than four.

1.2 The three decisions

Every learning system, from linear regression to a frontier language model, is defined by three choices. Make them explicit before you write any code, because they are usually made implicitly and then forgotten.

Decision one: the hypothesis class. What **shapes of rule** are you willing to consider?

This is the most consequential decision and the one most often made by accident. If you type `LogisticRegression()`, you have decided that the rule is a straight line. If you type `DecisionTreeClassifier()`, you have decided the rule is a set of yes/no questions about one feature at a time. Neither choice is neutral, and neither can be undone by more data.

IN PLAIN ENGLISH

The hypothesis class is **what the shop stocks**. If you walk into a shop that sells only straight rulers, no amount of searching will produce a compass. You can be the most determined shopper in the world; there are no curves in the building.

This is why “my model isn’t accurate enough, let me get more data” is sometimes exactly the wrong response. If the rule you need has a shape your model cannot make, more data changes nothing at all.

Decision two: the loss. How do you turn “this answer is wrong” into a **single number**?

You need one number because the adjusting step needs something to reduce. If the model predicted €12 and the true price was €15, is that a badness of 3? Of 9 (the square)? Of something else? The choice matters more than it looks: squaring makes one large error count for more than several small ones, which changes what the model ends up doing.

IN PLAIN ENGLISH

The loss is **the complaints procedure**. It decides which mistakes get taken seriously.

A shop that treats one very angry customer as worse than ten mildly annoyed ones will behave differently from a shop that adds up total annoyance. Neither is wrong. But they will end up with different policies, and if you never chose which shop you were running, you got one by accident.

Decision three: the optimiser. How do you search the space of allowed rules for a good one?

For essentially everything in this module, the answer is **gradient descent**, which is Chapter 11. It is the “adjust the guess slightly” step, done in a principled way.

WHY THIS MATTERS

These three decisions are the frame for the entire rest of the module, and being able to name them turns an intimidating paper into three familiar questions.

When Week 7 introduces transformers, the useful question is not “how does it learn?” (it learns the same way everything else does) but “**what hypothesis class does this give me, and why is that a good bet for language?**”. When Week 5 introduces physics-informed networks, the innovation is entirely in decision two: they add a term to the loss that penalises violating a physical law. When Week 9 covers efficient training, it is almost entirely about decision three.

Students who cannot name the three decisions experience each new architecture as a separate thing to memorise. Students who can experience them as variations on one theme.

1.3 Parameters and hyperparameters

Two words that get confused constantly. The distinction is simple and it is worth marks.

Parameters are the numbers **the algorithm** works out. Weights and biases. You do not choose them; you choose the procedure that produces them. A frontier language model has hundreds of billions of parameters. The network you will build in Chapter 14 has nine.

Hyperparameters are the numbers **you** choose, before training starts. How fast to adjust (the learning rate). How many layers. How deep a tree may grow. How strongly to penalise complexity.

IN PLAIN ENGLISH

Parameters are what the apprentice learns on the job. Hyperparameters are the instructions you gave the apprentice on day one: how quickly to change their mind, how much detail to pay attention to, when to stop.

Nobody learns the hyperparameters for you. If you did not choose them, you accepted whatever the library’s author guessed would suit most people, which may or may not suit you.

WHY THIS MATTERS

A large share of practical machine learning is **hyperparameter judgement**, and a striking proportion of failures, including published ones, are hyperparameter failures rather than conceptual ones. Chapter 12 is devoted to a single hyperparameter with the power to make a perfectly correct system learn absolutely nothing, silently.

There is also a Week 2 point here. A hyperparameter you did not choose is a decision made by nobody, with your name on the commit. Week 2, Chapter 7 made this argument about the decision threshold; it applies just as exactly to the learning rate and the tree depth.

1.4 Supervised, unsupervised, and everything else

Almost everything this week is **supervised**: every training example arrives with the right answer attached, and the loss compares your guess to that answer.

- **Supervised.** You have inputs and the answers. Predicting house prices from square footage; classifying email as spam. Weeks 3, 4, 7 to 9.
- **Unsupervised.** You have inputs and no answers, and the model must find structure by itself: grouping customers who behave similarly, or compressing data. Week 5.
- **Reinforcement.** No answers, but a reward signal that arrives late and tells you how well a whole sequence of decisions went. Behind AlphaGo (Week 1) and behind RLHF (Week 7).

IN PLAIN ENGLISH

Supervised learning is doing past exam papers **with** the marking scheme. Unsupervised learning is being handed a thousand unlabelled photographs and asked to sort them into piles that make sense, with nobody telling you what the piles should be. Reinforcement learning is learning chess by playing whole games and only being told at the end whether you won.

The training loop of the next chapter is the same in all three. Only the definition of “wrong” changes.

COMMON MISTAKE

“Unsupervised learning is what you use when you cannot be bothered to label the data.”

It is not a cheaper substitute for supervised learning; it answers a different question. Supervised learning asks **“what is the answer for this input?”** Unsupervised learning asks **“what is the structure of this data?”** If you actually need the first answer and only have unlabelled data, the honest response is usually to go and label some, not to switch paradigm.

QUIZ WATCH · WEEK 6

Very likely in some form: state the three decisions; distinguish parameters from hyperparameters with two examples of each; define supervised learning in one sentence and distinguish it from unsupervised. A common applied item gives you a failure scenario and asks **which of the three decisions** is at fault.

2 Reading the Mathematics

COVERS	A decoder for every piece of notation in this booklet
TIME	About 12 minutes; return to it whenever a formula stops you
IN BRIEF	The mathematics in this module is mostly ordinary arithmetic wearing formal clothes. This chapter takes the clothes off.

Formulas in machine learning look harder than they are, mostly because they compress a loop into a symbol. Nothing in this booklet requires mathematics beyond first-year calculus, and most of it requires only the ability to read a summation sign without flinching.

If you already read notation comfortably, skip this chapter. If you do not, it will save you more time than any other twelve minutes in the module.

2.1 The symbols, one at a time

Symbol	Says	Means in plain English
x	an input	one feature: a number describing the thing, like income or pixel brightness
$\mathbf{x}$	a vector of inputs	all the features for one example, in a list: [44, 50612, 29]
y	the true answer	what actually happened: the real price, or 1 for spam
$\hat{y}$	the predicted answer	what the model guessed. The hat always means “estimated”
$w, \mathbf{w}$	a weight, or all the weights	how much the model cares about each feature
b	the bias	a constant added at the end; where the line sits when every input is zero
θ	the parameters	a single name for all the weights and biases together
η	the learning rate	how big a step to take when adjusting. Greek letter “eta”
L	the loss	one number saying how badly the model is doing
$\sum_{i=1}^n$	sum over examples	“add this up for every one of the n training examples” , a for loop
$\sigma(z)$	the sigmoid	squash any number into the range 0 to 1, so it can be read as a probability
$\partial L / \partial w$	a partial derivative	“if I nudge w up a tiny bit, how much does L change?”
$\leftarrow$	assignment	“replace the thing on the left with the thing on the right” , as in code
$\mathbf{Ax}$	matrix times vector	do a whole batch of weighted sums at once, in one operation

IN PLAIN ENGLISH

Two symbols do almost all the intimidating work, and both are ordinary ideas.

$\sum_{i=1}^n z_i$ is a for loop that adds things up. In Python it is `sum(z[i] for i in range(n))`. That is genuinely all it is.

$\partial L / \partial w$ is a question, not an operation: **“if I increase w slightly, does the loss go up or down, and how steeply?”** If the answer is “up, steeply”, you should decrease w , and by a lot. That single sentence is the whole of Chapter 11.

2.2 Reading a formula out loud

Take the mean squared error loss, which looks like this:

$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Read it left to right, out loud, as instructions:

1. “The loss L is...”
2. “...one over n times...”, so we are going to **average** something over the n examples.
3. “...the sum over every example i ...”, a loop.
4. “...of $(\hat{y}_i - y_i)$...”, the guess minus the truth. The **error** on example i .
5. “...squared.”, so it is positive, and big errors count for much more than small ones.

Put together: **“take the error on each example, square it, and average.”** Four lines of Python:

```
errors = predictions - truth
loss = (errors ** 2).mean()
```

WHY THIS MATTERS

People who find machine learning papers impenetrable usually do not lack the mathematics. They lack the **habit** of translating notation into instructions as they read.

Do this deliberately for a month and it becomes automatic. Every formula in Weeks 4 to 12 (attention, diffusion, the loss of a physics-informed network) yields to the same procedure: read it left to right, out loud, as instructions, and name what each piece is doing. There is no formula in this module that survives that treatment.

WORKED EXAMPLE: Translating a formula you have not met

Here is the logistic regression loss, which appears in Chapter 5 and will look alarming the first time:

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

Read it as instructions. There is a **loop over examples** and a **divide by n** , so we are averaging something. Inside, there is a square bracket with two terms and a minus sign in front of everything.

Now notice that y_i is either 0 or 1. If $y_i = 1$, the second term is multiplied by $(1 - 1) = 0$ and vanishes, leaving $\log \hat{p}_i$. If $y_i = 0$, the first term vanishes, leaving $\log(1 - \hat{p}_i)$.

So the bracket is a switch. It says: **“if the true answer was yes, score the probability you gave to yes; if the true answer was no, score the probability you gave to no.”** Then take the log, average, and flip the sign so that bigger means worse.

In Python, with no notation at all:

```
p_true_class = np.where(y == 1, p, 1 - p)
loss = -np.log(p_true_class).mean()
```

It is not a hard idea. It was wearing a coat.

QUIZ WATCH · WEEK 6

Notation is not examined for its own sake, but every applied question requires reading it. If a formula in the quiz stops you, translate it into a loop and a sentence before attempting the question. That takes thirty seconds and it is almost always enough.

3 The Training Loop

COVERS	Forward, loss, backward, update, the four stages every model in this module runs
TIME	About 15 minutes; the spine of the week
IN BRIEF	Week 1's picture was a wave, Week 2's a pipeline. Week 3's is a loop with exactly four stages, and learning it now saves you nine weeks of confusion.

Everything that learns in this module runs the same four-stage loop. Learn it once and the rest of the semester becomes a series of substitutions into it.

1. **Forward.** Push the input through the model you currently have and produce a guess. Nothing clever happens here: it is arithmetic using the weights as they presently stand.
2. **Loss.** Compare the guess to the answer and reduce the disagreement to **one number**.
3. **Backward.** Work out how much **each individual weight** contributed to that number. This is the difficult stage. It has a name, **credit assignment**, and it is what stalled the field between 1969 and 1986.
4. **Update.** Nudge every weight a small distance in the direction that reduces the loss. One line of arithmetic.

Then repeat, often for millions of iterations.

IN PLAIN ENGLISH

You are learning to throw darts, blindfolded, with a friend calling out where each dart landed.

Forward: you throw, using whatever grip and angle you currently favour. **Loss:** your friend says “twenty centimetres low and a bit left”, one summary of how wrong you were. **Backward:** you work out which parts of your throw caused which part of the miss, the wrist angle mostly caused the height, the shoulder mostly caused the sideways drift. **Update:** you adjust each of those slightly. Then you throw again.

A neural network does exactly this, with a million adjustable joints instead of three, and it throws a few million darts.

3.1 Where the difficulty actually is

Three of the four stages are easy. Forward is arithmetic. Loss is a formula. Update is one line of subtraction.

Backward is the hard one, and it is worth understanding why before you meet the mechanism in Chapter 14. Your friend told you the dart landed twenty centimetres low. That is a single fact about the **outcome**. But your throw involved a wrist angle, a shoulder rotation, a release timing and a grip pressure. How much of the twenty centimetres was the wrist's fault?

For the last thing you did before releasing, this is answerable. For something that happened early in the movement, whose effect passed through everything after it, it is much harder. That is the credit assignment problem, and it is the reason a hidden layer was untrainable for seventeen years despite everyone knowing it was the answer.

WHY THIS MATTERS

This four-stage loop is the single most reusable thing in the module.

Every architecture in Weeks 4 to 12 changes the **first box** and leaves the other three nearly untouched. A convolutional network, a transformer, a state-space model and a diffusion model all run this loop, with the same backward stage and the same update rule.

So when you meet a new architecture (and you will meet six more), the useful first question is never “how does it learn?” It learns exactly the way everything else does. The useful question is “**what does its forward pass compute, and why is that a sensible assumption about this kind of data?**” That question is Week 4’s entire content, and it only makes sense if you already know the other three boxes are fixed.

3.2 The hillwalker

The analogy this booklet will use for the rest of the week, because every technical term maps onto a part of it.

EVERYDAY ANALOGY

You are on a mountainside in thick fog, trying to reach the valley floor. You cannot see the valley. You cannot see ten metres in any direction. But you can feel the slope under your feet, so you take a step downhill, and repeat.

The **gradient** is the slope under your feet. The **learning rate** is the size of your step. A **local minimum** is a hollow that is not the valley floor. **Momentum** is the tendency to keep going in the direction you were already going. The **loss surface** is the landscape itself.

And note what the walker never has: **a map**. Gradient descent is entirely local. It knows the slope where it stands and nothing else, not where the valley is, not whether a better valley exists two ridges over, not even whether it is nearly finished.

COMMON MISTAKE

“The model searches for the best answer.”

It does not search in any meaningful sense. It stands somewhere, feels which way is down, and steps. It has no picture of the space it is moving through, no memory of where it has been (unless you give it momentum), and no ability to consider a location it has not physically reached. Every surprising thing in Chapters 12 and 13 follows from taking that limitation seriously.

4 Linear Regression

COVERS	The simplest useful model, and the one you should always fit first
TIME	About 20 minutes
IN BRIEF	Predict a number as a weighted sum of the inputs. Every coefficient is a sentence you can say out loud, which turns out to matter enormously.

4.1 The model

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_dx_d + b$$

Multiply each feature by a weight, add them up, add a constant, and that is your prediction.

IN PLAIN ENGLISH

You are estimating what a second-hand car will sell for. You decide the price is:

€8,000, minus €600 for every year old, plus €40 for every horsepower, minus €0.05 for every kilometre on the clock.

That is a linear model, written out in English. The €8,000 is the **bias**, the starting point before you know anything. The €600, €40 and €0.05 are the **weights**. Learning the model means finding good values for those four numbers.

Each weight answers a question a human can read: **if this feature increases by one unit, and nothing else changes, how much does the prediction move?** That readability is not a minor convenience: Week 2 spent a chapter arguing it may be worth more than accuracy in high-stakes decisions.

4.2 The loss

$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Take each prediction, subtract the truth, square it, average over all examples. Squaring does two jobs: it makes errors positive so overestimates and underestimates cannot cancel out, and it punishes large errors disproportionately.

WORKED EXAMPLE: Two errors, two losses

Suppose your model makes two predictions and the errors are €10 and €10.

Squared errors: $10^2 + 10^2 = 200$.

Now suppose the errors are €0 and €20 instead, same total error of €20, distributed differently.

Squared errors: $0^2 + 20^2 = 400$.

Twice the loss for the same total error. Squaring means the model will work hard to avoid any single large mistake, even at the cost of several small ones. If your application genuinely prefers one big error to many small ones (and some do), squared error is the wrong choice, and mean absolute error is the usual alternative.

WHY THIS MATTERS

This is your first concrete example of the loss being a **decision about values**, which is the Week 2 idea arriving in a technical setting.

Squared error is not a law of nature. It is a statement that outliers matter a great deal. In house-price prediction that may be right. In predicting delivery times, where being an hour late once is far less damaging than being five minutes late constantly, it is probably wrong. Nobody will tell you this; the library default is squared error and it will run without complaint either way.

4.3 Fitting it

Find the weights that make the loss as small as possible. Chapter 11 explains the general method. For this particular model there is also an exact algebraic answer.

◉ DEEPER DETAIL: The closed form, and why almost nobody uses it

Linear regression is one of very few models in this module with an exact solution. Setting the derivative of the loss to zero and solving gives the **normal equations**:

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

No iteration, no learning rate, no convergence to worry about. Which raises a fair question: why does the rest of the module bother with gradient descent?

Three reasons, and they generalise. First, that matrix inversion costs roughly d^3 operations for d features, so it becomes impractical once you have thousands of features and impossible with millions. Second, it needs the entire dataset in memory simultaneously, which rules out anything web-scale. Third and most importantly, **it exists only for this model**. Add a single non-linearity and there is no closed form at all.

Gradient descent is slower here and it is the only method that generalises to everything else in the module. That is why it won, and it is a pattern worth noticing: the general method that is mediocre everywhere beats the perfect method that works in one place.

4.4 Why fit it first, always

✓ IN PRACTICE: The linear baseline

- It takes about two minutes and gives you a number every later model must beat.
- If your deep model beats it by one point, you have learned something important about the **problem**, not about the model.
- In the Week 3 lab, logistic regression scores 0.852 on a dataset deliberately built to defeat linear models, against 0.895 for the best of seven. That is **four points**. Know what four points cost you in transparency before you spend them.
- It is also a bug detector. If the linear baseline scores 0.99 on a problem you expected to be hard, you almost certainly have data leakage, and you have found it in two minutes rather than two weeks.

COMMON MISTAKE

“Linear models are for teaching; real work uses deep learning.”

Linear and logistic regression remain the workhorses of medicine, credit scoring, insurance and epidemiology, not because those fields are behind, but because in each of them somebody must be able to explain

a decision to a person affected by it. Week 2, Chapter 12 covered the legal machinery that makes this a requirement rather than a preference.

QUIZ WATCH · WEEK 6

Reliable: write down the linear model and its usual loss; explain what a single weight means in words; explain why squaring the error changes the model's behaviour and not just the scale; and give one reason the closed-form solution is not used in practice.

5 Logistic Regression

COVERS	The same straight line, doing a completely different job
TIME	About 18 minutes
IN BRIEF	Squash a weighted sum into a probability, change the loss, and a regression model becomes the most widely deployed classifier in the world.

Linear regression predicts a **number**. Very often what you want is a **category**: spam or not, will repay or will not, benign or malignant.

The obvious idea, predict a number and call anything above 0.5 a “yes”, half works and half fails. It fails because a weighted sum can produce any value at all, including -40 and 300 , and neither is interpretable as a probability.

5.1 The sigmoid

So we squash. The **logistic sigmoid** takes any number and returns something between 0 and 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \hat{p} = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$$

IN PLAIN ENGLISH

The sigmoid is a squashing function shaped like a stretched letter S.

Feed it a very negative number and it returns almost 0. Feed it a very positive number and it returns almost 1. Feed it zero and it returns exactly 0.5. In between it changes smoothly.

So the model computes a weighted sum exactly as before, call it the “score”, and then converts that score into a probability. A big positive score means “very likely yes”; a big negative score means “very likely no”; a score near zero means “genuinely uncertain”.

5.2 The loss

Squared error is a poor fit for probabilities, so classification uses **binary cross-entropy**:

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

Chapter 2 already translated this: it is a switch that scores the probability you assigned to whatever actually happened, takes the log, averages, and flips the sign.

WORKED EXAMPLE: Why the logarithm matters

Two models predict on a case where the true answer is **yes**.

Model A says “70% yes”. Its loss contribution is $-\log(0.7) = 0.357$.

Model B says “1% yes”, confidently, catastrophically wrong. Its loss is $-\log(0.01) = 4.605$.

That is **thirteen times** the penalty, for one case. Push further: a model that says “0.01% yes” scores $-\log(0.0001) = 9.21$, and a model that says exactly 0% scores **infinity**.

Compare with squared error, where the worst possible mistake scores 1. Cross-entropy punishes **confident wrongness** far more severely than mere wrongness, and that is deliberate. A system that is uncertain and wrong is doing its job badly. A system that is **certain** and wrong is dangerous, and the loss should say so.

WHY THIS MATTERS

This is the technical machinery behind Week 2's Chapter 9. Calibration, a score of 0.7 meaning a 0.7 chance, is something cross-entropy actively encourages, because the loss is minimised when your stated probabilities match reality.

It also explains why AI systems that state confident falsehoods are a specific engineering failure and not just an annoyance: something has gone wrong in the relationship between the model's stated confidence and its actual reliability, and the loss function was supposed to prevent exactly that.

5.3 It is still a linear classifier

COMMON MISTAKE

“Logistic regression is non-linear, because look at the S-curve.”

This is the single most common misunderstanding of the model, and it matters.

The **decision boundary**, the set of inputs where the model switches from predicting no to predicting yes, is where $\hat{p} = 0.5$, which happens exactly where $w \cdot x + b = 0$. That is the equation of a straight line (or a flat plane in higher dimensions).

The sigmoid bends the **output**, not the boundary. It changes how confident the model is as you move away from the line; it cannot make the line curve. So if your two classes are separated by a circle, logistic regression cannot express that, and the S-shaped curve in every textbook picture will not help you.

WHY THIS MATTERS

Knowing this tells you what to do when logistic regression underperforms, and the answer is not “train it longer”.

The options are: engineer better features (add x^2 or x_1x_2 as new columns, which lets a linear model draw a curve in the original space); or change the hypothesis class to something that bends natively. That is a Chapter 1 decision-one problem, and recognising it as such saves you from tuning a model that cannot possibly work.

QUIZ WATCH · WEEK 6

High-probability: write down the logistic model; explain what the sigmoid does in one sentence; explain why the model is called **linear** despite the curve; and explain why cross-entropy punishes confident errors more heavily than squared error would.

6 Decision Trees

COVERS	Twenty questions, played by a machine
TIME	About 18 minutes
IN BRIEF	A completely different shape of assumption from the straight line: yes/no questions about one feature at a time. Readable, flexible, and it will memorise your data completely if you let it.

6.1 The idea

A decision tree asks a series of yes/no questions about one feature at a time. **Is income above €32,000? If yes, is tenure above 14 months? If yes, predict “will not churn”.**

IN PLAIN ENGLISH

It is the game of twenty questions, or the flowchart a call-centre worker follows.

“Is the customer’s account more than a year old? If yes, go to question 4. If no, go to question 9.” Each answer narrows things down, and at the bottom of every path is a decision.

A doctor’s triage protocol is a decision tree. So is a tax form. Humans have used this structure for centuries precisely because it is easy to follow and easy to check.

6.2 How the machine chooses the questions

This is the only genuinely technical part. At every node the algorithm tries **every feature** and **every possible threshold**, and picks the split that leaves the two resulting groups as **pure** as possible.

“Pure” means containing mostly one class. Two standard measures:

$$\text{Gini} = 1 - \sum_k p_k^2, \quad \text{Entropy} = - \sum_k p_k \log_2 p_k$$

where p_k is the proportion of class k in the node.

WORKED EXAMPLE: Computing Gini impurity by hand

A node contains 10 customers: 5 who churned, 5 who did not. So $p_1 = p_2 = 0.5$.

$$\text{Gini} = 1 - (0.5^2 + 0.5^2) = 1 - 0.5 = 0.5$$

That is the **maximum** impurity for two classes: a coin flip, no information at all.

Now suppose a split produces a node with 9 churners and 1 non-churner: $p_1 = 0.9$, $p_2 = 0.1$.

$$\text{Gini} = 1 - (0.81 + 0.01) = 0.18$$

Much lower, much purer. And a node with 10 churners and 0 others gives $1 - 1 = 0$: perfectly pure, nothing left to ask.

The algorithm simply tries every candidate split and keeps whichever produces the lowest weighted average impurity in the two children. That is all “learning” means for a tree.

The procedure is **greedy**: it takes the best split available right now and never reconsiders. This is not optimal, a slightly worse split now might enable a much better one later, but finding the genuinely optimal tree is computationally intractable, so greedy is what everyone uses.

6.3 What trees buy and what they cost

Advantages	Limitations
No feature scaling needed: “is income above €32,000” does not care what units anything else is in. Handles numeric and categorical features together. Copes with missing values. Non-linear boundaries. And you can read the rules out loud .	Boundaries are stuck at right angles to the axes, so a diagonal boundary costs a staircase of splits. Small changes in the data can produce a completely different tree. And left unchecked it will keep splitting until every leaf holds exactly one training point.

IN PLAIN ENGLISH

The right-angle limitation is worth picturing. Suppose the real rule is “**reject if income plus savings is below €40,000**”, a diagonal line on a plot of income against savings.

A tree cannot ask about income **plus** savings; it can only ask about one at a time. So it approximates the diagonal with a staircase: “income below 10k? reject. Otherwise, income below 20k and savings below 20k? reject. Otherwise, income below 30k and savings below 10k? reject...” It gets there eventually, using dozens of splits to approximate one line, and each step is fitted to the particular points it happened to see.

6.4 The memorisation problem

Left to itself, a tree will keep splitting until every leaf is pure, which it can always achieve by isolating each training point in its own leaf. At that moment it has 100% training accuracy and has learned nothing generalisable.

The lab measures this:

Model	Train	Test	Gap
Decision tree (depth 3)	0.915	0.876	0.039
Decision tree (unlimited)	1.000	0.857	0.143

The unlimited tree gets **every training point right** and is beaten on unseen data by a tree one fifth its size.

WHY THIS MATTERS

This table is the most important thing in the chapter, and it generalises far beyond trees.

Perfect training accuracy is not an achievement; it is a symptom. Any model with enough freedom can memorise its training data, and a model that has memorised has learned the **specific rows** rather than the process that generated them.

Chapter 9 makes this formal. But notice already what it does to how you report results: a single accuracy figure is not evidence of anything unless you say which data it was measured on and what the other number was.

The remedy is to limit the tree’s freedom: cap the depth, require a minimum number of samples per leaf, or grow it fully and then prune back. All three are hyperparameters, and all three are your decision.

QUIZ WATCH · WEEK 6

Expect: explain how a split is chosen and what Gini impurity measures, ideally with a small calculation; give two advantages and two limitations; explain why an unlimited tree can score 1.000 on training data and worse than a shallow tree on test data.

7 Random Forests

COVERS	Two hundred mediocre trees beat one good one
TIME	About 12 minutes
IN BRIEF	The standard fix for the previous chapter's problem, and a genuinely surprising result: averaging models that each overfit produces one that does not.

7.1 The idea

Stop trying to build one good tree. Build hundreds of deliberately **different** ones and average their votes.

The differences are manufactured two ways. Each tree is trained on a **bootstrap sample**: draw n rows at random with replacement, so each tree sees a slightly different dataset. And at every split, each tree is only allowed to consider a **random subset of the features**, so trees cannot all latch onto the same dominant variable.

IN PLAIN ENGLISH

Ask one expert and you get one opinion, with all of that expert's particular blind spots baked in.

Ask two hundred experts, each of whom read a slightly different set of case files and was forbidden from considering certain evidence, and then take a vote. Every individual is worse than your one careful expert. The **committee** is better, because their idiosyncratic mistakes point in different directions and cancel, while their shared correct judgements reinforce.

This works only if the mistakes are genuinely uncorrelated. If all two hundred experts trained at the same institution and share a blind spot, the vote confidently reproduces it.

7.2 Why averaging works

Recall the bias-variance idea from Chapter 9 (which you may want to read first): error comes from being the wrong shape (**bias**) and from being too sensitive to the particular sample you saw (**variance**).

A fully grown tree has low bias, it can represent almost anything, and enormous variance, because move a few points and you get a different tree. Averaging many high-variance, low-bias models leaves the bias roughly where it was and **drives the variance down**. Whether it works depends entirely on whether the errors are uncorrelated, which is exactly what the bootstrap sampling and random feature subsets are engineering.

WHY THIS MATTERS

The forest creates an honest complication for the story in Chapter 9, and you should know it before the quiz rather than after.

In the lab, the random forest scores **1.000 on training data** (it memorises completely, just like the unlimited tree) and scores 0.890 on test data, the second-best result of seven models.

So “memorises the training set” is **not by itself** a diagnosis. What matters is whether the memorisation is correlated across the ensemble. The crude version of the overfitting story (“perfect training accuracy is bad”) is a good first approximation and it is wrong in this specific, common case. Knowing where a rule of thumb breaks is worth more than the rule.

7.3 The price

Week 2's price. You have traded a model you could read aloud to a refused applicant for a committee of two hundred that you cannot. There is no free lunch here, it is a genuine trade, and Week 2, Chapter 12 argued it should be made explicitly and measured rather than assumed.

✓ IN PRACTICE: Where forests are the right first choice

- Tabular data with mixed types and a few thousand to a few million rows. Forests and gradient-boosted trees still routinely beat neural networks in this setting.
- When you need a strong baseline fast and cannot afford to tune. Forests are famously forgiving of default hyperparameters.
- When you want **feature importance** for free, with the caveat from Week 2, Chapter 12: importance tells you what the model used, not whether it should have.

QUIZ WATCH · WEEK 6

Likely: explain what a random forest does differently from a single tree, naming both sources of randomness; explain why averaging reduces variance; and explain why a forest can memorise its training data and still generalise well.

8 Support Vector Machines

COVERS	Where exactly should the line go, and what to do when no line will work
TIME	About 20 minutes
IN BRIEF	A sharper question than logistic regression asks, and one of the most elegant ideas in the field: getting the benefit of a huge transformation without ever performing it.

8.1 The better question

If two classes can be separated by a line, **infinitely many** lines separate them. Logistic regression returns one that fits. The support vector machine asks: which line leaves the **widest empty corridor** between the classes?

$$\max_{w,b} \frac{2}{\|w\|} \quad \text{subject to} \quad y_i(w \cdot x_i + b) \geq 1 \quad \text{for every } i$$

IN PLAIN ENGLISH

You are painting a line down the middle of a road with parked cars on both sides. You could paint it anywhere in the gap. But the sensible place is **exactly in the middle**, as far from the cars on both sides as possible, because then a driver who wanders slightly still has room.

The margin is the width of that clear space. Maximising it means putting your boundary as far from **both** classes as you can, on the reasoning that a boundary with room to spare is more likely to survive contact with data it has not seen.

The width of the corridor is the **margin**. The points sitting exactly on its edge are the **support vectors**, and they are the only ones that matter: delete every other training point and the solution is unchanged.

WORKED EXAMPLE: Why “only the support vectors matter” is remarkable

Train an SVM on 10,000 points and you might find 40 support vectors. Throw away the other 9,960 rows, retrain, and you get **the identical boundary**.

This is genuinely useful. It makes the model compact, you only need to store the support vectors, and it explains why SVMs are robust to points far from the boundary: a customer who is obviously creditworthy contributes nothing to where the line goes.

It also has an uncomfortable corollary that Week 2 would recognise. The model’s entire behaviour is determined by a handful of borderline cases. If those particular cases are unrepresentative, or if the borderline population differs systematically between groups, the boundary inherits that and nothing in the aggregate accuracy will reveal it.

8.2 Soft margins

Real data overlaps, so the hard constraint is relaxed: points may sit inside the corridor, or on the wrong side of it, at a penalty controlled by a hyperparameter C .

- **Small C** : mistakes are cheap. The model tolerates violations and produces a wide, smooth boundary. More bias, less variance.
- **Large C** : mistakes are expensive. The model contorts to classify every training point correctly. Less bias, more variance.

It is the capacity dial of Chapter 9 wearing a different name, and you will meet the same dial under a third name (regularisation strength) and a fourth (tree depth).

8.3 The kernel trick

The elegant part. If no line works in the space you have, map the data into a higher-dimensional space where one does.

IN PLAIN ENGLISH

Picture a bullseye: a cluster of red dots in the middle, surrounded by a ring of blue dots. On a flat page, no straight line separates them. Ever.

Now lift the page into three dimensions and push the middle upward, so distance from the centre becomes height. The red dots rise into a peak; the blue ring stays low. Slide a flat sheet of card horizontally between them, and it separates them perfectly.

A flat sheet in three dimensions is exactly what the SVM knows how to find. The data did not change; the **space** did.

The difficulty is that useful mappings are enormous, sometimes infinite-dimensional, and computing them is impossible. The trick is that the SVM's mathematics never needs the mapped points themselves, only **dot products between pairs of them**. A **kernel function** computes those dot products directly:

$$K(x, x') = \exp(-\gamma \|x - x'\|^2) \quad (\text{the RBF kernel})$$

EVERYDAY ANALOGY

You want to know whether two people would get on. The honest method is to introduce them and watch. The kernel trick is a mutual friend who can simply tell you the answer.

You get the quantity you needed. The meeting never took place. That is the whole trick: the expensive thing was never done, and the answer is exact rather than approximate.

WHY THIS MATTERS

This is what dominated the 1990s and much of the 2000s, and it is Week 1's "statistical turn" in one page. It matters for two reasons beyond history.

First, it has not gone away. In the Week 3 lab, on a dataset built to be awkward, the RBF-kernel SVM is the **best of seven models** (0.895 on held-out data) and has the **smallest train-test gap of all of them** (0.020). A method from the 1990s beats the neural network. Age is not a measure of usefulness.

Second, the idea generalises. "Transform the data into a space where the problem is easy" is exactly what a hidden layer does in Chapter 14, what an embedding does in Week 7, and what a physics-informed prior does in Week 5. The SVM does it with a fixed function chosen by you; a neural network **learns** the transformation. That contrast is one of the cleanest ways to explain what deep learning actually added.

COMMON MISTAKE

"The kernel trick maps the data into a higher-dimensional space."

Almost, and the imprecision costs marks. It computes **what the dot products would be** if you had done the mapping. The points are never mapped, the high-dimensional space is never constructed, and for the RBF

kernel it could not be, because it is infinite-dimensional. The point of the trick is that the expensive step is skipped entirely.

QUIZ WATCH · WEEK 6

Expect: define the margin and explain why maximising it is desirable; explain what a support vector is and what follows from only they mattering; explain the kernel trick in two sentences **without using the word kernel**; and say what C controls.

9 Capacity: Underfitting and Overfitting

COVERS	How much model is too much, and the one number that tells you
TIME	About 25 minutes; the most examinable chapter in the booklet
IN BRIEF	Training error is not evidence of anything on its own. The gap between training and test error is the diagnosis, and this chapter is why you never report a single accuracy figure again.

9.1 The two ways to fail

Underfitting. The model is too restricted to represent the pattern even in principle. Training error is high, and test error is equally high.

Overfitting. The model is rich enough to fit the **noise** as well as the signal, and it does. Training error is very low; test error is high. The model learned facts about the particular rows it happened to see rather than about the process that produced them.

IN PLAIN ENGLISH

A student is preparing for an exam using ten past papers.

The **underfitting** student has skimmed one paper and learned one very general rule: “always answer C”. They do badly on the practice papers and equally badly on the real exam. Their problem is not effort; it is that what they learned is too crude to be right about anything.

The **overfitting** student has memorised all ten past papers perfectly, every question, every answer, in order. They score 100% on the practice papers. Then the real exam asks slightly different questions and they collapse, because they learned **the papers** rather than **the subject**.

The second student looks much better than the first right up until the moment that matters. That is the entire problem: overfitting is invisible if you only measure on the data you trained on.

9.2 Watching it happen

The lab fits polynomials of increasing flexibility to twenty-six noisy points sampled from a sine wave:

Degree	Train MSE	True MSE	What happened
1	0.204	0.228	Underfit. A straight line cannot bend. Bad everywhere, equally.
4	0.033	0.021	About right. Captures the shape, ignores the noise.
18	0.012	1499.8	Overfit. Passes through every point; useless between them.

Look very carefully at the degree-18 row. It has the **lowest training error of the three**. Its error against the truth is roughly seventy thousand times worse than the middle model’s. If training error had been your criterion, you would have chosen it and been confident.

WORKED EXAMPLE: Why the number is so absurdly large

A degree-18 polynomial has nineteen coefficients and there are twenty-six points. With that much freedom it can thread through nearly every point exactly, so its training error is tiny and genuinely so.

But between the points it is unconstrained, and high-degree polynomials do something violent when unconstrained: they swing to enormous values. Between two data points 0.03 apart, the fitted curve might travel from -40 to +60 and back.

Those excursions never touch a training point, so they cost nothing on the training set. They are catastrophic on any new point that happens to land there. Hence 0.012 against 1499.8.

This is why “it fits the data well” is not the same claim as “it is a good model”, and it is why the picture in the lab is worth more than the numbers.

WHY THIS MATTERS

This chapter is the reason for one of the most important professional habits in the field: **never report training accuracy as though it were a result.**

Report the **gap**. It takes two numbers instead of one, it is visible at a glance, and it is the single cheapest diagnostic available. In the lab’s seven-model comparison the gap column immediately identifies the unlimited tree (0.143) as having memorised, while the RBF SVM (0.020) has not.

There is also a direct Week 2 connection. A model that memorised its training data will fail unpredictably in deployment, and it will fail **hardest on the cases least like the training data**, which, as Week 2, Chapter 4 argued at length, are disproportionately the under-represented ones. Overfitting is not only a technical failure; it is a failure whose costs land unevenly.

9.3 The bias-variance decomposition

The formal statement, and one of the few pieces of theory in this module that changes how you actually work. For squared-error loss, a model’s expected error on a new point splits into three parts:

$$\mathbb{E}[(\text{error})^2] = \underbrace{(\text{bias})^2}_{\text{wrong shape}} + \underbrace{\text{variance}}_{\text{too sensitive}} + \underbrace{\sigma^2}_{\text{irreducible noise}}$$

IN PLAIN ENGLISH

Imagine four people throwing darts at a board, and we watch each of them throw twenty darts.

High bias, low variance: all twenty darts land tightly grouped, ten centimetres below the bullseye. Consistent, and consistently wrong. This is the straight line fitted to a curve: it will make the same mistake every time, no matter how much data you give it.

Low bias, high variance: the darts average out to the bullseye, but they are scattered all over the board. Any single throw could be anywhere. This is the degree-18 polynomial: give it a different sample and it produces a wildly different answer.

Irreducible noise: the room has a draught. Nobody’s darts will ever land perfectly, and no amount of skill fixes it. Pretending this component is zero is exactly what the degree-18 model did, it treated the draught as signal and modelled it.

Increasing capacity reduces bias and increases variance. That trade is the reason the classical test-error curve is U-shaped, with a sweet spot in the middle. The table above is that curve, with three points on it.

COMMON MISTAKE

“Bias here means the same thing as bias in Week 2.”

It does not, and Week 2, Chapter 2 warned you about exactly this word. Statistical bias means an estimator that is systematically off, a technical property with no moral content. Societal bias means systematic disadvantage to a group of people.

A model can have low statistical bias and cause serious societal harm. A model can have high statistical bias and be perfectly fair. When you write “bias” in an answer, say which one.

◉ DEEPER DETAIL: Where the classical picture stops being the whole story

Everything above is the textbook account, it is what the Week 6 quiz examines, and it is also incomplete. You should know why before you meet the exception.

Very large modern networks routinely have **far more parameters than training examples**, interpolate the training data exactly, and generalise well anyway. On the U-shaped curve, that should be impossible.

The phenomenon is called **double descent**: as capacity grows past the point where the model can exactly fit the data, test error rises as the classical picture predicts, and then, as capacity keeps growing, falls again, sometimes below the classical sweet spot.

The current understanding is that hugely over-parameterised models trained by gradient descent are **implicitly biased toward simple solutions** among the many that fit the data perfectly. So raw capacity is the wrong axis; what matters is the effective complexity of the solution the optimiser happens to land on. Notice that the random forest in Chapter 7 is a small everyday instance of the same puzzle.

The classical account remains the right first model of what is going on, and it earns its place. By Week 7 you will need the caveat.

9.4 What to do about it

✓ IN PRACTICE: Six remedies, cheapest first

- **Hold out a test set before you look at anything**, and do not touch it until the very end. Every time you glance at it and adjust something, you leak a little information into your choices. Week 1’s benchmark-culture lesson, applied in private.
- **Cross-validate**. Split the data five ways, train five times holding out a different fifth, average. Far more reliable than one split when data is scarce.
- **Reduce capacity**. Shallower tree, fewer layers, lower polynomial degree, smaller C .
- **Regularise**. Add a penalty for large weights: L_2 (ridge) shrinks them all toward zero, L_1 (lasso) pushes many to exactly zero and thereby selects features. Same effect as reducing capacity, achieved without changing the architecture.
- **Get more data**. The only remedy that reduces variance without increasing bias, and usually the most expensive.
- **Stop early**. Watch validation loss during training and stop when it turns upward. Free, and remarkably effective.

WHY THIS MATTERS

Notice that five of the six remedies work by **restricting the model**, and only one works by getting more data. This is counter-intuitive for people arriving from software engineering, where more capability is usually better.

In machine learning, unconstrained capability is a liability. The skill is choosing exactly how much freedom the model needs and then denying it any more. That framing carries directly into Week 5, where physics supplies the constraint, and into Week 9, where the constraint is imposed by hardware.

QUIZ WATCH · WEEK 6

Near-certain, in several forms: define underfitting and overfitting with the train/test signature of each; state the bias-variance decomposition and explain all three terms; name four remedies; and, the classic applied item, given a table of train and test scores, say which model you would deploy and justify it. The degree-18 row is the favourite: lowest training error, worst model on the page.

10 The Perceptron and Its Wall

COVERS	1958, 1969, and the promise Week 1 left here
TIME	About 20 minutes
IN BRIEF	A machine that learns from its own errors, a genuine convergence theorem, and a four-point problem that stopped an entire research programme.

Week 1, Chapter 6 left this chapter a debt. Here it is, paid in full.

10.1 One unit

$$\hat{y} = \begin{cases} 1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

Multiply each input by a weight, add them up, and if the total clears a threshold, output 1.

IN PLAIN ENGLISH

A committee with one chairperson and no discussion.

Each member (feature) has a vote, and the votes are weighted, some members' opinions count for three times others'. Add up the weighted votes. If the total is above the line, the motion passes.

"Learning" means adjusting how much each member's vote is worth, based on how often the committee's decisions turned out to be wrong.

10.2 The learning rule

$$\mathbf{w} \leftarrow \mathbf{w} + \eta(y - \hat{y})\mathbf{x}$$

Read it as instructions: when the machine is wrong, move the weights toward the right answer, in proportion to the input that caused the error.

WORKED EXAMPLE: One step of perceptron learning

Two features, current weights $\mathbf{w} = [0.5, -0.2]$, bias $b = 0$, learning rate $\eta = 0.1$.

An example arrives: $\mathbf{x} = [1, 1]$, true answer $y = 1$.

Forward: $0.5(1) + (-0.2)(1) + 0 = 0.3$. That is above zero, so $\hat{y} = 1$. Correct, the error $(y - \hat{y})$ is zero, so nothing changes. Good.

Next example: $\mathbf{x} = [1, 3]$, true answer $y = 1$.

Forward: $0.5(1) + (-0.2)(3) = 0.5 - 0.6 = -0.1$. Below zero, so $\hat{y} = 0$. **Wrong.**

Update: the error is $(1 - 0) = +1$, so

$$\mathbf{w} \leftarrow [0.5, -0.2] + 0.1 \times 1 \times [1, 3] = [0.6, 0.1]$$

Check the same example again: $0.6(1) + 0.1(3) = 0.9$. Above zero. Now correct.

Notice **why** the second weight moved most: the update is proportional to the input, and the second feature had value 3. The rule blames the inputs that were loudest.

10.3 The theorem, and its condition

Rosenblatt proved something real: the **perceptron convergence theorem**. If the training data can be separated by a straight line, this procedure will find such a line in a **finite** number of steps.

That is a genuine guarantee, not empirical, not usually, but provably. It is the source of the 1958 headlines Week 1 quoted.

KEY IDEA

Read the condition again. **If the data can be separated by a straight line.** Everything that happened over the following two decades lives inside that “if”.

10.4 XOR

Four points. Output 1 when the two inputs **differ**, 0 when they match.

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

IN PLAIN ENGLISH

Draw a square. Put a dot at each corner. Colour the bottom-left and top-right corners **blue**, and the other two **amber**.

Now try to draw one straight line with both blue dots on one side and both amber dots on the other.

You cannot. The blue dots are on one diagonal and the amber ones on the other, so any line separating them would have to bend or cross itself. This is not a matter of trying harder or searching longer: **there is no such line**, and thirty seconds with a pencil proves it more convincingly than any amount of algebra.

Minsky and Papert made this precise in 1969, and Week 1 recorded the consequence: the result was read far more broadly than it warranted, and neural network research lost most of its funding for a decade.

What the lab shows, and why it is more convincing than the proof

Run `perceptron()` in `scratch_net.py` on XOR for two hundred epochs. Its accuracy never exceeds **3 out of 4**, and it does not creep upward.

The rule is not converging slowly. It is **cycling**, fixing one point breaks another, and it goes round for ever, because there is no weight vector to converge to. The failure is structural rather than a matter of patience or tuning.

Watching it fail for two hundred epochs teaches something the proof does not: what a structurally impossible problem **looks like** from the inside. It looks exactly like a problem that needs more training, right up until you realise it never will.

WHY THIS MATTERS

This episode is the single best illustration in the module of a lesson that outlives the technical content.

The theorem was **correct**. The critique was **valid**. And the conclusion the field drew, that neural networks were a dead end, was wrong, because the result reached only the simplest version of the idea and there was an escape route the whole time.

Week 1's discipline for reading criticism was: **which version of the idea does this reach, and is there an escape route?** Here the answer was "one layer only" and "yes, a hidden layer". Nobody applied that discipline, and the cost was a decade.

You will be asked to evaluate confident claims about the limits of AI throughout your career. This is the case study.

The escape route was known almost immediately: a **hidden layer**. The obstacle was that nobody knew how to train one, because nobody had a way to work out how much of the output error each hidden weight was responsible for. That is Chapter 14, and it took until 1986.

QUIZ WATCH · WEEK 6

Very likely: write out the XOR truth table and explain in two sentences why no single perceptron can compute it; state what the convergence theorem guarantees **and what it assumes**; and explain precisely what took seventeen years, it was not the idea of a hidden layer, which was obvious.

11 Gradient Descent

COVERS The walk downhill, and the three ways of taking a step
TIME About 20 minutes
IN BRIEF One line of arithmetic, repeated. Everything that learns in this module learns this way, including every model you will meet in Weeks 4 to 12.

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}$$

That is the whole algorithm. Read it as instructions: “**replace each parameter with itself, minus the learning rate times its derivative.**”

IN PLAIN ENGLISH

You are in fog on a mountainside, trying to reach the valley.

You cannot see anything, but you can feel which way the ground slopes under your feet. So you take a step in the downhill direction. Then you feel again, the slope may have changed, and take another step. Repeat until the ground is flat.

$\partial L / \partial \theta$ is the slope you feel. η is the size of your step. The minus sign is you deciding to go **down** rather than up.

11.1 Why the derivative is exactly the right object

For a single parameter, the derivative answers precisely the question you need: **if I nudge this weight up by a tiny amount, does the loss go up or down, and how steeply?**

- Derivative **positive**: increasing the weight increases the loss. So decrease it.
- Derivative **negative**: increasing the weight decreases the loss. So increase it.
- Derivative **large in magnitude**: this weight matters a lot right now. Move it further.
- Derivative **near zero**: this weight barely affects anything here. Leave it roughly alone.

The minus sign in the update handles the first two cases automatically, and the magnitude handles the last two. One line covers all four.

WORKED EXAMPLE: Descent by hand, on the simplest surface

Take $f(x) = x^2/2$. Its derivative is $f'(x) = x$. Start at $x = 4$ with $\eta = 0.1$.

step	x	gradient	new $x = x - 0.1x$
0	4.000	4.000	3.600
1	3.600	3.600	3.240
2	3.240	3.240	2.916
3	2.916	2.916	2.624

Two things to notice. Each step is **smaller** than the last, because the gradient shrinks as you approach the minimum, descent slows down automatically as it arrives, without being told to.

And the update simplifies beautifully here: $x \leftarrow x - \eta x = (1 - \eta)x$. Each step multiplies x by 0.9. That observation is the whole of Chapter 12, and you just derived it.

With millions of parameters, the gradient is simply the vector of all those individual answers, and every parameter is adjusted according to its own, all at once.

KEY IDEA

Gradient descent is **purely local**. It knows the slope where it currently stands and nothing else, not where the minimum is, not whether a better valley exists over the next ridge, not even whether it is nearly finished. Everything surprising in Chapters 12 and 13 is a consequence of that single limitation.

11.2 Batch, stochastic, mini-batch

The gradient is defined over the whole training set. Computing it exactly means looking at every example before taking one step, which is expensive. There are three options.

Variant	How	Trade-off
Batch	Use every training example, then take one step.	The most accurate estimate of the true slope, and unusably slow: one step per full pass over the data, and the data must fit in memory.
Stochastic (SGD)	One example per step.	Very fast per step, and very noisy, each example gives a wobbly estimate of the true slope. The noise is not purely a cost: it helps the walker rattle out of shallow hollows.
Mini-batch	32 to 512 examples per step.	Accurate enough, fast enough, and it is the shape a GPU wants. This is what everybody actually uses.

IN PLAIN ENGLISH

You are trying to work out which way a crowd is drifting.

Batch: survey all fifty thousand people, then take one step. Extremely accurate, and by the time you finish the crowd has moved.

Stochastic: ask one person and step immediately. Fast, and that person might be walking to the toilet, so you get jostled about. But you take thousands of steps in the time the batch method took one, and the errors largely cancel.

Mini-batch: ask a hundred people. Accurate enough, still fast. Everyone does this.

WHY THIS MATTERS

There is a lesson in **why** mini-batch won that recurs throughout this module, so it is worth stating explicitly.

Mini-batch gradient descent is not mathematically superior to the alternatives. It won because it **matches the hardware**: a GPU is efficient when it can do a few hundred identical operations simultaneously, and wasted when asked to do one.

This pattern, the method that fits the machine beats the method that is theoretically nicer, appears again in Week 1's account of why the transformer displaced the LSTM (attention parallelises, recurrence does not), and again in Week 8, where state-space models are designed around what hardware does cheaply. Chapter 17 makes the general case.

COMMON MISTAKE

“SGD’s noise is a problem to be eliminated.”

The noise is genuinely useful, and pure batch descent is not obviously better even where you can afford it. A noisy walker gets bounced out of shallow hollows that a perfectly accurate one would settle into, and there is good evidence that the noise biases training toward flatter, better-generalising minima. Chapter 13 develops this.

12 The Learning Rate

COVERS	One hyperparameter, four failure modes, and a band narrower than anyone expects
TIME	About 22 minutes; the core of the lab
IN BRIEF	Two orders of magnitude separate “crawling” from “diverging”, nothing in your code tells you which side you are on, and getting it wrong is the single largest source of wasted time in this field.

12.1 The four failure modes

The lab runs gradient descent on the simplest possible surface, $f(x) = x^2/2$, starting at $x = 4$, for forty steps, at five learning rates:

η	final x	final loss	What happened
0.02	1.783	1.589	Crawling. Correct direction, and still descending when the budget ran out.
0.10	0.059	0.0017	Converged smoothly.
0.90	4e-40	8e-80	Converged smoothly, and far faster.
1.98	1.783	1.589	Oscillating. Bouncing across the minimum, losing almost nothing per bounce.
2.05	28.16	396.5	Diverging. Each overshoot larger than the last.

IN PLAIN ENGLISH

Back to the hillwalker, and the step size.

Too small: you shuffle downhill in ten-centimetre steps. You are going the right way and you will be out all night.

About right: you stride down, arrive, stop.

Slightly too large: you take fifty-metre strides, overshoot the bottom, land partway up the far slope, turn round, overshoot back. You are bouncing between the two sides, and you do get slightly lower each time, but you will be there for ever.

Too large: each stride is **bigger** than the last. You bounce higher and higher up alternating slopes until you leave the mountain entirely. In code this shows up as the loss becoming `inf` and then `nan`.

12.2 Two observations that matter more than the table

First: rows one and four have identical losses and opposite diseases. One is creeping downhill in tiny steps. The other is leaping across the valley and back. If you had reported only the final loss you could not tell them apart, and the fixes are opposite. Row one needs a **larger** learning rate; row four needs a **smaller** one. This is why you plot the loss curve rather than reading the final number.

Second: the boundary is exact, and you can derive it. For this surface the update is

$$x \leftarrow x - \eta x = (1 - \eta)x$$

so the magnitude of x shrinks only when $|1 - \eta| < 1$, which means $0 < \eta < 2$. At $\eta = 1$ it lands exactly on the minimum in a single step. Above 2 it diverges.

 WORKED EXAMPLE: Checking the boundary by hand

With $\eta = 1.98$: each step multiplies x by $(1 - 1.98) = -0.98$.

The sign flips every step, that is the oscillation, and the magnitude shrinks by only 2% each time. After forty steps: $4 \times 0.98^{40} = 1.78$. Which is exactly the number in the table.

With $\eta = 2.05$: each step multiplies by $(1 - 2.05) = -1.05$. Sign flips, magnitude **grows** 5% each step. After forty steps: $4 \times 1.05^{40} = 28.2$. Also exactly the table.

Nothing here is empirical. The code is confirming arithmetic you can do on paper, which is precisely why this surface was chosen for the lab.

WHY THIS MATTERS

Now generalise the uncomfortable part. On this two-line surface you can derive the safe range exactly. On a real network with ten million parameters and a loss surface nobody has ever seen, **you cannot**.

There is no warning, no exception, no message. Above the threshold, the loss becomes nan and the model learns nothing; slightly below it, training appears to work and converges to something poor. Both look like “the model isn’t very good”.

This is why the standard advice is to **sweep the learning rate before tuning anything else**. It is ten minutes, and it removes the largest single source of wasted time in applied machine learning. Every experienced practitioner has lost a week to this once, and only once.

 IN PRACTICE: Finding it in practice

- **Sweep, do not guess.** Try powers of ten from $1e-5$ to $1e-1$, a few hundred steps each, and plot the loss curves on one axis.
- **Read the shape of the curve.** Flat means too small. Noisy but descending is usually fine. Rising, spiking, or nan means too large.
- **Use a schedule.** Large steps early when you are far away, small steps as you approach. Cosine decay and step decay are the common choices, and they exist because the ideal step size changes during training.
- **Prefer an adaptive optimiser.** Adam and its relatives keep a separate effective step size per parameter and are far more forgiving of a bad initial choice. They are the sensible default. They do not abolish the problem: Adam still has a learning rate, and it can still be wrong.

COMMON MISTAKE

“My loss went to nan, so there must be a bug in my data.”

Sometimes. But by far the most common cause is a learning rate above the divergence threshold, and it is a thirty-second thing to rule out: divide the learning rate by ten and rerun. Check that before you spend an afternoon auditing your data pipeline.

QUIZ WATCH · WEEK 6

Near-certain: name the four failure modes with their signatures; for $f(x) = x^2/2$ **derive** the range of η for which descent converges; and explain how two runs can have identical final losses and opposite problems. The derivation is a favourite because it is short, exact, and shows whether you understand the update rule or have memorised it.

13 Local Minima, Saddles, and Momentum

COVERS	What is actually on the loss surface, and why high dimensions are kinder than intuition suggests
TIME	About 18 minutes
IN BRIEF	Local minima are real, and in very high dimensions much rarer than you would expect. Saddle points and plateaus are the real obstacle, and the fixes differ.

13.1 The problem, demonstrated

Run the **same** algorithm at the **same** learning rate on a bumpy one-dimensional surface, from three different starting points:

start	final x	final loss	
-2.5	-1.044	6.656	found the good valley
1.0	-1.044	6.656	found the good valley
5.5	4.097	8.948	settled 34% worse, and cannot tell

The third run has, by its own criterion, finished. Every direction from where it stands goes up. The gradient is zero. It has no way whatsoever of knowing that a better valley exists, because, Chapter 11, it has no map.

IN PLAIN ENGLISH

Our hillwalker in fog reaches a spot where the ground is flat in every direction. Feeling around with their feet, everything slopes up.

From the walker's point of view, this is the bottom. They have succeeded.

In fact they are in a small hollow two hundred metres up the mountainside, and the actual valley is over the next ridge. Nothing available to them (no amount of care, patience or feeling about) can reveal this. The only things that would help are starting somewhere else, or having enough momentum to roll straight through the hollow.

13.2 Why deep networks suffer less than you would expect

In one dimension a local minimum requires the surface to curve upward in **two** directions. In d dimensions it requires the surface to curve upward in **all d directions simultaneously**.

With a hundred million parameters, the chance that every single one of a hundred million directions curves upward at a randomly encountered flat point is vanishingly small. Almost always, at least one direction still goes down.

IN PLAIN ENGLISH

Picture a mountain pass, a saddle between two peaks.

Stand at the middle of the pass. Walk toward either peak and you go **up**. Walk along the pass in either direction and you go **down**. The ground is flat where you stand, but it is not a bottom; there is a way out, and it is obvious once you look along the right axis.

In two dimensions, hollows are common and passes are special. In a hundred million dimensions it is exactly the other way round: for a point to be a genuine bottom, **every one of a hundred million** directions must go up, which almost never happens. What you meet instead are passes, flat, slow, and escapable.

What you get instead are **saddle points**: flat in many directions, up in some, down in others. These slow training badly, because the gradient is nearly zero and progress stalls for long stretches, but they are escapable.

KEY IDEA

The received wisdom that “deep learning works despite local minima” is not quite right, and the accurate version is more useful: **local minima are rare in very high dimensions; saddle points and plateaus are the real obstacle.**

This matters practically because the fixes differ. For a local minimum you need a different starting point or enough momentum to roll through. For a saddle you mostly need **patience and noise**: SGD’s randomness will eventually knock you off it, and momentum will carry you along the downhill axis once you find it.

13.3 Momentum

The smallest useful improvement to the update rule:

$$v \leftarrow \beta v - \eta \frac{\partial L}{\partial \theta}, \quad \theta \leftarrow \theta + v$$

Instead of stepping purely according to the current slope, keep a **velocity**: a decaying memory of the steps you have already taken.

IN PLAIN ENGLISH

Stop imagining a walker taking discrete steps and start imagining a **ball rolling downhill**.

A ball builds speed on a long consistent slope, so it gets to the bottom faster. A ball is not stopped by a small dip; it rolls straight through. And in a narrow valley where the walker would zigzag from wall to wall, a ball’s side-to-side motions cancel while its downhill motion accumulates.

The price (and it is a real price) is that a ball arriving at the bottom with speed will roll some way up the far side before turning round.

The lab measures exactly that price:

momentum β	steps to loss < 0.01	best loss	final loss
0.0	32	0.000026	0.000026
0.5	12	0.000000	0.000000
0.9	5	0.000031	0.011594

Read the last row honestly. Momentum 0.9 reaches the neighbourhood of the minimum in **five** steps rather than thirty-two, six times faster, and then **rattles around it**, finishing worse than it was at its best.

WHY THIS MATTERS

Momentum is not free speed. It is speed traded for overshoot, which is precisely why it is normally paired with a decaying learning rate: build speed while you are far away, then slow down as you arrive.

This is also your first encounter with a pattern that will recur for the rest of the module. Every fix in this field creates the next problem. ReLU fixes vanishing gradients and creates dying units. Momentum fixes slow convergence and creates overshoot. Compression fixes model size and creates the disparate impact of Week 2, Chapter 7.

Knowing the name of the problem your fix just created is most of the skill, and it is what separates someone who can tune a model from someone who can only run one.

QUIZ WATCH · WEEK 6

Expect: explain what a local minimum is and why they are rarer in high dimensions; distinguish a saddle point from a local minimum **and say why the distinction changes what you would do**; write the momentum update and explain the velocity term in one sentence; state one cost of high momentum.

14 Backpropagation

COVERS	Credit assignment, the chain rule, and the eight lines that took seventeen years
TIME	About 30 minutes; the centre of the week
IN BRIEF	Backpropagation is not a learning algorithm. It is an efficient way to compute one gradient. Gradient descent does the learning, and confusing the two is the most common error in exam answers on this topic.

14.1 The problem

The network's output was wrong by 0.31. The network has nine weights.

How much of that 0.31 is the fault of each one?

For the final layer this is answerable: those weights touch the output directly, and their influence is immediate. But a weight three layers back affects the output only through everything in front of it. Working out its share of the blame is the **credit assignment problem**.

IN PLAIN ENGLISH

A dish comes back from the restaurant table. It is too salty.

Whose fault? The person who salted it at the end, obviously a suspect, and easy to check. But also possibly the person who made the stock four hours ago, whose saltiness passed through the reduction, the sauce and the final seasoning, being concentrated at each stage. And the person who chose the salted butter. And the supplier.

To improve the kitchen you need to know **how much** each person should change what they do. Blaming everyone equally is useless. Blaming only the last person is wrong. You need to trace the saltiness backwards through every step, working out how much each stage **amplified or diluted** what came before it.

That is backpropagation. And it is genuinely hard, which is why a hidden layer was untrainable for seventeen years despite everyone knowing it was the answer.

14.2 The solution: the chain rule

The mechanism is the chain rule from first-year calculus, applied systematically and in the right order.

The chain rule says: if a affects b , and b affects c , then the effect of a on c is the **product** of the two individual effects.

$$\frac{\partial c}{\partial a} = \frac{\partial c}{\partial b} \times \frac{\partial b}{\partial a}$$

IN PLAIN ENGLISH

Turning a tap changes the flow of water. The flow of water changes how fast the bath fills.

If turning the tap one notch doubles the flow, and doubling the flow halves the filling time, then turning the tap one notch halves the filling time: you **multiply** the effects along the chain.

A neural network is a long chain of exactly this kind. Each layer takes what the previous one produced and does something to it. To find out how a weight in layer one affects the final loss, you multiply together the effect at every stage in between.

14.3 The network, named

Our two-layer network, with the names used throughout this booklet and in the lab code:

$$z_1 = \mathbf{X}\mathbf{W}_1 + \mathbf{b}_1, \quad a_1 = \sigma(z_1), \quad z_2 = a_1\mathbf{W}_2 + \mathbf{b}_2, \quad a_2 = \sigma(z_2), \quad L = \text{mean}(a_2 - y)^2$$

Read it as a pipeline: multiply the input by the first weights (z_1), squash (a_1), multiply by the second weights (z_2), squash (a_2), compare to the truth (L). The z values are **pre-activations**; the a values are **activations**.

The gradient for a first-layer weight is the product of every local effect along that chain:

$$\frac{\partial L}{\partial \mathbf{W}_1} = \frac{\partial L}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial \mathbf{W}_1}$$

Each factor is easy on its own. The insight is to compute them **right to left**, keeping each partial result for the layer behind, which turns what looks like an enormous amount of repeated work into a single backward pass costing about the same as the forward pass.

WORKED EXAMPLE: Why right-to-left matters so much

Suppose you computed each weight's gradient independently, left to right, chasing its effect forward through the network. For a network with L layers and n weights per layer, you would repeat almost the same multiplication chain for every one of the nL weights.

Computing right to left, you calculate the “blame arriving at layer k ” **once**, and every weight in layer k uses it. The shared work is done a single time.

The saving is roughly a factor of the number of parameters. For a nine-weight network that is the difference between trivial and trivial. For a nine-billion-parameter network it is the difference between possible and **never**.

This is why backpropagation is described as an **efficient** algorithm rather than a correct one. Computing these gradients naively was always possible; computing them fast enough to be useful was not.

14.4 The two passes

Forward: compute the prediction, and **keep every intermediate value** (z_1, a_1, z_2, a_2). You will need them on the way back.

Backward: walk the network in reverse, multiplying derivatives. Each layer receives the accumulated blame from the layer in front, uses it to compute the gradient for its own weights, and passes an appropriate share further back.

WHY THIS MATTERS

“Keep every intermediate value” sounds like a footnote and has enormous practical consequences.

It is why **training a network needs far more memory than running one**. Running a model forward, you can discard each layer's output as soon as the next layer has consumed it. Training, you must hold all of them until the backward pass arrives.

This single fact explains a great deal you will meet later: why you can run a model on a laptop that you could not possibly train there; why batch size is limited by memory rather than compute; why gradient checkpointing

(recomputing activations instead of storing them) exists; and why Week 9 budgets training and inference costs completely separately. One line in this chapter, and it shapes the economics of the entire field.

14.5 The code

Here is all of it, from the lab:

```
def backward(self, X, Y, lr):
    n = len(X)
    d_a2 = 2.0 * (self.a2 - Y) / n          # how wrong the output is
    d_z2 = d_a2 * sigmoid_d(self.a2)        # back through the activation
    d_W2 = self.a1.T @ d_z2                 # blame the weights that fed it

    d_a1 = d_z2 @ self.W2.T                 # send the blame BACK a layer
    d_z1 = d_a1 * sigmoid_d(self.a1)
    d_W1 = X.T @ d_z1

    for param, grad in ((self.W1, d_W1), (self.W2, d_W2), ...):
        param -= lr * grad                  # the update from Chapter 11
```

KEY IDEA

Line five is backpropagation. `d_a1 = d_z2 @ self.W2.T` is the moment the blame crosses a layer boundary, travelling backwards through the same weights that carried the signal forwards.

Everything else in that function is bookkeeping. That single idea, that you can push error **backwards** through the same connections that pushed the signal forwards, is what the field could not find between 1969 and 1986.

KEY IDEA

Backpropagation is **not** a learning algorithm. It computes one gradient, efficiently. **Gradient descent is what learns.**

Conflating the two is the most common error in exam answers on this topic. Keeping them separate also makes several later topics much clearer: when Week 9 discusses training cost, it is talking about the backward pass; when it discusses inference cost, only the forward pass exists.

14.6 Check your gradient

Two lines that will save you two days:

```
numeric = (loss_at(w + eps) - loss_at(w - eps)) / (2 * eps)
```

Perturb one weight by a tiny amount, observe how the loss **actually** changes, and compare against what your derivation claimed. In the lab these agree to about 10^{-12} .

IN PLAIN ENGLISH

You have written down a formula for how steep a hill is at a particular point. To check it, walk one centimetre forward, measure how much your height changed, and divide.

If your formula says the slope is 0.4 and the measurement says 0.4, your formula is right. If the formula says 0.4 and the measurement says -0.9, you have made an algebra mistake, and you have found it in five seconds instead of five days.

WHY THIS MATTERS

A wrong derivative **does not raise an error**. Nothing crashes. Instead you get a model that trains slowly, or plateaus early, or almost works, and you will spend two days blaming your data, your learning rate and your architecture, in that order.

Modern frameworks compute gradients for you, so you will rarely derive one by hand. But the moment you write a custom layer or a custom loss (and by Week 10 several of you will), you are back here. Check it once, numerically, and proceed with confidence.

14.7 And then XOR falls over

scratch_net.py, 8000 epochs, about a second of compute:

input	target	predicted
[0. 0.]	0	0.0268
[0. 1.]	1	0.9684
[1. 0.]	1	0.9688
[1. 1.]	0	0.0305

But what the hidden layer **learned** is more interesting than the fact that it worked. These are the two hidden units' outputs for each of the four inputs:

[0. 0.]	-> [0.001, 0.914]
[0. 1.]	-> [0.081, 0.024]
[1. 0.]	-> [0.084, 0.020]
[1. 1.]	-> [0.873, 0.000]

IN PLAIN ENGLISH

Look at the middle two rows. The inputs (0, 1) and (1, 0), the two that should output 1, have been mapped to almost **the same place**: roughly (0.08, 0.02).

Meanwhile (0, 0) has been pushed to (0.00, 0.91) and (1, 1) to (0.87, 0.00), opposite corners.

So the hidden layer has rearranged the four points. In the original square the two “yes” points sat on opposite diagonal corners, which is what made them unseparable. In the new arrangement they are practically on top of each other, with the two “no” points far away on either side.

And **now a straight line works**, which is exactly what the output layer is. The impossible problem was made possible by moving the points.

KEY IDEA

A hidden layer does not buy you **more parameters**. It buys you a **change of coordinates**: a new space, learned from the data, in which the problem happens to be easy.

WHY THIS MATTERS

This is the single most important sentence in Week 3, because it is the bridge to the rest of the module.

Week 4 takes it and asks: what if we design the coordinate change deliberately? A convolutional layer is a change of coordinates that assumes nearby pixels are related. A recurrent layer assumes order matters. Attention assumes any element may relate to any other.

Week 5 takes it further: what if the new coordinates come from **physics** rather than from data? That is what a physics-informed network is.

Week 7 is entirely about one particular learned coordinate system, embeddings, in which words with similar meanings end up near each other.

Chapter 8's kernel trick did the same thing with a **fixed** transformation that you chose. The difference deep learning made is that the transformation is **learned**. That contrast is the cleanest one-sentence answer to “what did deep learning actually add?”, and you now have it from first principles.

QUIZ WATCH · WEEK 6

Near-certain, in several forms: state the credit assignment problem; explain what backpropagation computes **and explicitly what it does not do**; write the chain rule for a two-layer network; explain why the forward pass must store its intermediate values **and one consequence of that**; describe a gradient check; and explain what a hidden layer buys you.

15 Activations and the Vanishing Gradient

COVERS	Why the non-linearity matters, and why the field abandoned the elegant one
TIME	About 18 minutes
IN BRIEF	Without a non-linearity, depth is an expensive way to multiply matrices. With the wrong one, depth does not train at all.

15.1 Why any non-linearity at all

Stack two linear layers with nothing between them:

$$W_2(W_1x) = (W_2W_1)x = Wx$$

The composition of two linear maps is a linear map. A hundred layers, still a line.

IN PLAIN ENGLISH

Scaling a recipe by 2, then by 3, is the same as scaling it by 6. You could have done it in one step.

Stacking multiplications gets you nothing that a single, larger multiplication would not have given you. If every layer of your network just multiplies, then a hundred layers is an expensive way to do what one layer does.

Squashing the numbers between the multiplications is what breaks that. Double the recipe, then **cap every quantity at 500g**, then triple it, now you cannot collapse the steps, because the cap did something no multiplication can undo.

WHY THIS MATTERS

This is the shortest complete argument for why activation functions exist, and it is worth being able to give it in two lines under exam conditions.

It also tells you what to check when a deep network performs no better than a shallow one. If someone has accidentally built a network with no activations, or with activations that have saturated into effectively linear behaviour, this is exactly what you would see, and no amount of adding layers will fix it.

15.2 The three you need

Function	Form	Behaviour
Sigmoid	$1/(1 + e^{-z})$	Smooth, bounded to $(0, 1)$, readable as a probability. Its derivative peaks at 0.25 and approaches zero at both ends.
tanh	$\tanh(z)$	Same shape, centred on zero, bounded to $(-1, 1)$. Zero-centred outputs help optimisation slightly. Still saturates.
ReLU	$\max(0, z)$	Not smooth, not bounded, not differentiable at zero. Gradient is exactly 1 where active and exactly 0 where not.

15.3 The vanishing gradient

Backpropagation **multiplies** a derivative at every layer (Chapter 14). Sigmoid's derivative is at most 0.25, and much smaller once a unit saturates.

WORKED EXAMPLE: Doing the multiplication

Take the best possible case: every sigmoid unit sitting exactly at its steepest point, derivative 0.25.

Through **two** layers, the gradient reaching the first is scaled by $0.25^2 = 0.0625$.

Through **five** layers: $0.25^5 = 0.00098$.

Through **ten** layers: $0.25^{10} \approx 0.00000095$, roughly one in a million.

And that is the **best** case. In practice units saturate, derivatives fall well below 0.25, and the true figure is far smaller.

So the first layer of a ten-layer sigmoid network receives a gradient about a millionth the size of the last layer's.

With a shared learning rate, the last layers train and the early layers essentially do not move from their random initial values.

This is the **vanishing gradient problem**, and it is a large part of why depth did not work before about 2010. The network was not too big or too slow; the learning signal was simply not reaching the front of it.

ReLU's gradient is exactly 1 wherever the unit is active, so it does not shrink as it propagates. That is the entire argument, and it is why a function that looks like an insult to the mathematics replaced two elegant ones.

IN PLAIN ENGLISH

Passing a message down a line of people, where each person only whispers a quarter as loudly as they heard it.

By the tenth person the message is inaudible. They know a message came, but not what it said, so they carry on doing whatever they were doing. That is a sigmoid network's first layer.

ReLU is the same line of people, but each repeats the message at exactly the volume they heard it, or, if they were not listening at all, stays silent. The people who **are** listening hear it perfectly, however long the line.

KEY IDEA

ReLU's own failure mode has a name too: **dying ReLU**. A unit pushed permanently into negative territory has gradient exactly zero for every input, receives no learning signal ever again, and is dead for the rest of training. Leaky ReLU, which gives a small slope for negative inputs, exists precisely for this.

Chapter 13's pattern again: every fix creates the next problem, and knowing the name of the one you just created is most of the skill.

COMMON MISTAKE

"The vanishing gradient means the gradient is small, so just use a bigger learning rate."

It does not help, and it usually makes things worse. The problem is not that all gradients are small; it is that they are **very unevenly sized across layers**, enormous at the output, microscopic at the input. Raise the learning rate and the last layers, which were training fine, now diverge, while the early ones still barely move.

The fix has to change the **ratio**, not the scale. That means a different activation, better initialisation, or architectural changes such as the skip connections that ResNet introduced in 2015, which Week 1 recorded and which exist for exactly this reason.

QUIZ WATCH · WEEK 6

Reliable: explain in one line why stacking linear layers without activations is pointless; name three activations with one property each; explain the vanishing gradient problem **including why multiplication causes it**, ideally with a number; say why ReLU helps and name its own failure mode.

16 PyTorch, and What It Is Doing for You

COVERS Autograd, the five-line training loop, and the trap that does not raise an error
TIME About 20 minutes
IN BRIEF The framework you will use every week from here to Week 12. Read Chapter 14 first: this chapter is only interesting once you know what it replaced.

16.1 Three things a framework gives you

Automatic differentiation. You write the forward pass; PyTorch records every operation you perform, building a **computational graph**, then walks that record backwards applying the chain rule.

IN PLAIN ENGLISH

It is not doing algebra. It never looks at your formula and differentiates it symbolically the way you would on paper.

It keeps a **receipt**. Every time you multiply, add or squash a tensor, PyTorch writes down what you did and to what. When you call `loss.backward()`, it reads that receipt from the bottom up, applying the chain rule at each entry.

This is why it works on code with `if` statements and loops in it: it does not need to understand your program, only to record what it actually did this time.

You can verify it on something you can differentiate in your head:

```
x = torch.tensor(2.0, requires_grad=True)
f = 3 * x ** 2 + 2 * x
f.backward()
x.grad          # 14.0, and by hand: f'(x) = 6x + 2, so f'(2) = 14
```

Hardware. `model.to("cuda")` and the same code runs on a GPU. Week 9 spends two hours on what that line costs in energy and money, and how to shrink it.

Components. Layers, losses, optimisers and data loaders that are tested, fast, and, importantly for reading papers, named the same way everywhere.

16.2 The five lines

```
for epoch in range(epochs):
    optimiser.zero_grad()      # 1. forget the previous step's gradients
    pred = model(X)            # 2. forward: compute the prediction
    loss = loss_fn(pred, Y)     # 3. measure: how wrong is it?
    loss.backward()             # 4. backward: assign blame to every weight
    optimiser.step()           # 5. update: take one step downhill
```

Lines 2 to 5 are exactly the four stages of Chapter 3. Learn them in this order; you will write them every week until Week 12.

16.3 Line for line against what you wrote

<code>scratch_net.py</code> (numpy)	<code>torch_intro.py</code>
<code>self.z1 = X @ self.W1 + self.b1</code>	<code>nn.Linear(2, h)</code>

<code>sigmoid(z)</code>	<code>nn.Sigmoid()</code>
<code>np.mean((a2 - y) ** 2)</code>	<code>nn.MSELoss()</code>
the entire <code>backward()</code> method	<code>loss.backward()</code>
<code>param -= lr * grad</code>	<code>optimiser.step()</code>
nothing: numpy has no hidden state	<code>optimiser.zero_grad()</code>

WHY THIS MATTERS

This table is the reason the lab makes you write the numpy version first, and it is worth being explicit about the payoff.

When `loss.backward()` misbehaves in Week 8, and it will, in the form of exploding gradients or a loss that will not move, you will know it is eight lines of chain rule rather than an oracle. You will know that the forward pass stored activations, so a memory error means your batch is too large. You will know that a nan propagates backwards from wherever it first appeared.

People who learned PyTorch first and the mathematics never experience these as unrelated mysteries. That is the entire difference this chapter exists to make.

16.4 The trap

Look at the last row of the table. `zero_grad()` has **no analogue** in the version you wrote by hand, and that is exactly why it catches people.

PyTorch **accumulates** gradients into `.grad` rather than replacing them. Omit `zero_grad()` and every step descends on the running sum of every gradient computed since the program started.

IN PLAIN ENGLISH

A whiteboard on which you write your calculation of which way to step.

You would expect it to be wiped between calculations. It is not. Each new calculation is written **on top of** the last, and what you read off is the sum of everything ever written there.

Step two: you are using the sum of two calculations. Step fifty: the sum of fifty. The number gets larger and less related to the current situation with every step, and nothing warns you, because adding numbers to a whiteboard is not an error.

It does not raise an exception. It does not warn. The lab runs the same four hundred epochs twice:

```
with optimiser.zero_grad()      final loss 0.244
WITHOUT zero_grad()            final loss 0.250
```

Neither crashed. One of them learned.

KEY IDEA

The accumulation behaviour is deliberate and occasionally essential: it is how you simulate a large batch on a small GPU, by accumulating gradients across several mini-batches before stepping.

A sensible default for advanced use, and a trap for everyone else. That is the general shape of a framework: it makes the common case short and the uncommon case possible, and the price is that the short version has behaviour you must know about.

✓ IN PRACTICE: Habits worth forming now

- **Write the training loop out longhand** for the first few weeks rather than using a wrapper. You are learning the loop, not the library.
- **Set seeds** (`torch.manual_seed`) and record them. Week 2's argument about accountability applies to your own results first.
- **Print the loss every few epochs.** If it is not falling, stop and find out why. Do not wait for the run to finish.
- **Count your parameters:** `sum(p.numel() for p in model.parameters())`. It is the single best check that the model you built is the model you intended, and it takes one line.
- **Overfit a tiny subset deliberately.** Take ten examples and train until the loss is essentially zero. If your model cannot memorise ten examples, it has a bug, and you have found it in thirty seconds rather than after a six-hour run.

17 Why It Waited Until 2012

COVERS Backpropagation worked in 1986. Deep learning arrived in 2012. This chapter is the gap.
TIME About 12 minutes
IN BRIEF Week 1's three ingredients, seen from inside the training loop.

Everything in Chapters 11 to 15 was known and published by the late 1980s. LeNet was reading cheques in production in 1989. So what took another twenty-three years?

Data. Gradient descent estimates the direction of steepest descent from examples. With a thousand images you can fit a small network; with fourteen million you can fit a deep one without it simply memorising, which is Chapter 9's argument, applied at scale. ImageNet arrived in 2009.

Compute. Training is millions of small multiply-and-add operations with no argument about the order, which is precisely the workload a graphics card was built for. CUDA made that hardware programmable for general purposes in 2007. Note that mini-batch sizes are chosen to suit the hardware, not the mathematics.

A handful of engineering fixes. ReLU in place of sigmoid (Chapter 15). Better weight initialisation, so signals neither explode nor vanish at the start. Dropout as regularisation. Data augmentation.

IN PLAIN ENGLISH

A recipe for bread has existed for four thousand years. Industrial bakeries did not.

What was missing was not the recipe. It was flour at scale, ovens that could run continuously, and a handful of unglamorous process fixes. When those arrived, bread production changed enormously, and **the recipe was the same recipe**.

2012 is the industrial bakery, not the invention of bread.

WHY THIS MATTERS

Getting this story right matters more than it might seem, for two reasons.

First, it is the honest version, and the honest version is more useful. The romantic version has a lone genius having an idea in 2012. The true version has an old algorithm meeting two supply curves, and it tells you where to look for the next such moment: not at a clever idea nobody has had, but at a capability that has just become cheap.

Second, it is the module's thesis in miniature. Week 1 argued that the current wave got three things right and is now hitting the limits of the brute-force version. Weeks 5, 8 and 9 are all about the same manoeuvre from the other direction: instead of waiting for more supply, **build better assumptions into the model** so you need less of it. AlexNet needed fourteen million labelled images. A physics-informed network in Week 5 can beat a data-driven baseline with a few hundred points, because it was told the governing equation instead of having to infer it.

KEY IDEA

AlexNet contained **almost no new mathematics**. It was 1986's algorithm, fed 2009's data, running on 2007's silicon, with a handful of engineering fixes. Week 1's Venn diagram, seen from inside the training loop: each region was a **supply**, and the centre was an assembly job.

18 A Field Guide: Seven Questions for Any Model You Train

COVERS The week, operationalised

TIME About 10 minutes, then use it forever

IN BRIEF Week 1 gave seven questions for a headline, Week 2 seven for anything you ship. These are the seven for anything you train, and you will use them most often.

1. **What shape of answer can this model represent, and does the truth have that shape?** (Chapters 4 to 8.) If the boundary is curved and your model draws lines, more data will not help. This is decision one from Chapter 1, and it is the question people skip.
2. **What is the loss, and what does it say I care about?** (Chapters 4, 5.) Squared error is a decision to punish large errors disproportionately. Cross-entropy is a decision to punish confident wrongness severely. Neither is neutral, and neither was chosen by you unless you chose it.
3. **What is my train-test gap, and did I look at it before celebrating?** (Chapter 9.) A single accuracy number is not a result. Two numbers and their difference is.
4. **Is my learning rate in the band between crawling and diverging?** (Chapter 12.) Sweep it before tuning anything else. It is the largest single source of wasted time in this field, and it costs ten minutes to rule out.
5. **Did I check the gradient once, numerically?** (Chapter 14.) Only relevant if you wrote a custom layer or loss, but then, always, because a wrong derivative fails silently.
6. **Would a simpler model have done?** In the lab, an RBF-kernel SVM beats the neural network and has the smallest train-test gap of all seven models. Fit the simple baseline and know what the complicated one cost you.
7. **Which hyperparameters did I choose, and can I defend each in a sentence?** If the answer is “it was the default”, that is a decision made by nobody with your name attached, which is Week 2’s argument, arriving in a new place.

TRY IT YOURSELF

Standing exercise for the semester: take any model you train and answer all seven in under two hundred words **before** you report a result to anybody. By Week 6 it is quiz revision. By Week 12 it is a section of your project’s critical brief, and it is marked.

19 The Revision Core: The Whole Week on Two Pages

If you memorise nothing else for Week 6, memorise this chapter.

19.1 The loop

Forward (guess) → **Loss** (score the guess with one number) → **Backward** (assign blame to every weight) → **Update** (step downhill) → repeat. Weeks 4 to 12 change the **first box** and leave the rest almost alone.

19.2 The three decisions

Hypothesis class (what shapes of answer are allowed) · **loss** (how a wrong answer is scored) · **optimiser** (how the space is searched). Parameters are learned by the algorithm; **hyperparameters** are chosen by you and nobody else.

19.3 The four model families

Family	Assumption	Watch out for
Linear / logistic	a straight boundary	cannot bend, ever. But readable, fast, and the right baseline. Logistic regression is a linear classifier despite the S-curve
Trees / forests	splits at right angles, one feature at a time	an unlimited tree memorises: 1.000 train, 0.857 test. A forest also hits 1.000 train and generalises fine
SVM	the widest empty corridor; or a line in a space you never visit	kernel and C both need choosing. Still frequently the best model on tabular data
Neural networks	compose simple bends until it fits	needs the most data, gives the least readability

19.4 Capacity

Underfit: high train error, high test error, wrong shape. **Overfit**: low train error, high test error, memorised the noise. **Report the gap**. Degree-18 polynomial: train MSE 0.012, true MSE 1499.8, and the **lowest training error of the three**. Bias-variance: error = bias² + variance + noise, wrong shape, too sensitive, and the part nobody can remove.

19.5 XOR and the hidden layer

No straight line separates XOR, so the perceptron never exceeds 3 of 4 no matter how long it trains: the failure is structural. One hidden layer **re-represents** the four points so that a line does work. The escape route was known in 1969; **training** it took until 1986. A hidden layer buys a change of coordinates, not just more parameters. Stacked linear layers with no activation collapse to a single linear layer.

19.6 Gradient descent

$\theta \leftarrow \theta - \eta \partial L / \partial \theta$. Purely local: it knows the slope where it stands and nothing else. Batch / stochastic / mini-batch, and mini-batch won because it **fits the hardware**. For $f(x) = x^2/2$ the update is $x \leftarrow (1 - \eta)x$, so it converges only for $0 < \eta < 2$. Four failure modes: crawling, converged, oscillating, diverging, and crawling and

oscillating can produce **identical losses**. Local minima are rare in high dimensions; **saddle points** are the real obstacle. Momentum is a decaying memory of past steps: six times faster to the neighbourhood, and it overshoots.

19.7 Backpropagation

The chain rule applied right to left, reusing partial results, which is what makes it **efficient** rather than merely correct. It computes a gradient; **gradient descent does the learning**. The forward pass must store intermediates, which is why training needs far more memory than inference. $d_{a1} = d_{z2} @ W2.T$ is the line where blame crosses a layer boundary. Sigmoid's derivative maxes at 0.25, so through ten layers the gradient scales by at most 10^{-6} : the **vanishing gradient**. ReLU's gradient is exactly 1 where active, and 0 where dead.

19.8 Ten one-line lessons

1. Machine learning is fitting a function to data by minimising a loss. Everything else is detail.
2. Choosing a model is choosing an assumption; when the assumption matches reality you need less data.
3. Perfect training accuracy is a symptom, not an achievement, unless you are averaging an ensemble.
4. The loss function is a values statement, and squaring the error is a decision about outliers.
5. The learning rate is not a detail: two orders of magnitude separate crawling from diverging, and the code will not tell you which you have.
6. A hidden layer is a learned change of coordinates.
7. Backpropagation computes a gradient; it does not learn.
8. Every fix creates the next problem: ReLU and dying units, momentum and overshoot.
9. `loss.backward()` replaced eight lines of chain rule, not the idea behind them.
10. AlexNet was 1986's algorithm on 2009's data and 2007's silicon.

QUIZ WATCH · WEEK 6

The Week 6 quiz draws its Week 3 items from exactly this chapter: the loop, the three decisions, the four families and their assumptions, under- versus overfitting with the gap, why XOR needs a hidden layer, what backpropagation computes, and the learning-rate failure modes. If you can reconstruct this chapter onto two blank pages from memory, Week 3 revision is done.

20 Glossary in Plain English

Term	Plain-English meaning	Ch.
Hypothesis class	The set of shapes of answer a model is allowed to produce. What the shop stocks.	1
Loss function	How a wrong answer becomes one number. The complaints procedure.	1
Parameter	A number the algorithm learns: a weight or a bias.	1
Hyperparameter	A number you choose before training. Nobody learns these for you.	1
Supervised learning	Learning from examples that come with the right answer attached.	1
Forward pass	Push the input through and produce a guess, keeping every intermediate value.	3
Credit assignment	Working out how much each individual weight contributed to the error.	3, 14
Bias (the parameter)	The constant added at the end. Where the line sits when every input is zero.	4
Mean squared error	Average of the squared errors. Punishes one big mistake more than several small ones.	4
Logistic regression	A weighted sum squashed to a probability. Still a linear classifier.	5
Sigmoid	An S-shaped squasher mapping any number into the range 0 to 1.	5
Cross-entropy	The classification loss. Punishes confident wrong answers severely.	5
Gini impurity	How mixed the classes are in a tree node. Zero when pure.	6
Greedy	Takes the best option available now and never reconsiders. What tree-building does.	6
Random forest	Hundreds of deliberately different trees, averaged. Overfits individually, generalises collectively.	7
Bootstrap sample	Draw n rows at random with replacement, so each tree sees a different dataset.	7
Margin	The width of the empty corridor between the classes. The SVM maximises it.	8
Support vector	A point on the corridor edge. Delete any other point and nothing changes.	8
Kernel trick	Getting the dot products of a high-dimensional mapping without ever doing the mapping.	8
Underfitting	Too little capacity: high train error and high test error. Wrong shape.	9
Overfitting	Too much capacity: low train error, high test error. Memorised the noise.	9
Train-test gap	The difference between the two. The single cheapest diagnostic there is.	9
Bias-variance	Error splits into wrong-shape, too-sensitive, and irreducible noise.	9
Regularisation	Penalising large weights to reduce capacity without changing the architecture.	9
Perceptron	A weighted sum and a threshold. Provably converges if a separating line exists.	10
XOR	The four-point problem no single perceptron can solve, and the reason the first neural era ended.	10
Gradient	The direction of steepest increase of the loss. Descent goes the other way.	11
Mini-batch	32 to 512 examples per step. Won because it fits the hardware.	11
Learning rate	How far to step. Too small crawls, too large diverges, and the good band is narrow.	12

Local minimum	A hollow where every direction goes up. Rarer in high dimensions than intuition suggests.	13
Saddle point	Flat in many directions, up in some, down in others. The real obstacle in deep networks.	13
Momentum	A decaying memory of past steps. Faster down long slopes; overshoots at the bottom.	13
Backpropagation	The chain rule applied right to left, reusing partial results. Computes a gradient; does not learn.	14
Chain rule	If a affects b and b affects c , multiply the two effects together.	14
Gradient check	Comparing your derivative against a numerical one. Two lines; saves two days.	14
Activation function	The non-linearity between layers. Without it, depth collapses to one linear layer.	15
Vanishing gradient	Derivatives below 1 multiplied layer after layer shrink toward zero, so early layers stop learning.	15
ReLU	$\max(0, z)$. Gradient exactly 1 where active, so it does not vanish. Can die permanently.	15
Autograd	PyTorch keeping a receipt of your operations and replaying it backwards. Not algebra.	16
<code>zero_grad()</code>	Clearing accumulated gradients. Omit it and nothing errors; the model just learns nonsense.	16

21 Question Bank: Practising for Week 6

Answers are not printed. Every question is answerable from a named chapter, and hunting the answer is the revision.

21.1 Short answer: two to three sentences each

1. State the three decisions behind every learning system. (Ch. 1)
2. Distinguish parameters from hyperparameters, with two examples of each. (Ch. 1)
3. Explain why “get more data” cannot fix a hypothesis class that is the wrong shape. (Ch. 1)
4. Translate $L = \frac{1}{n} \sum (\hat{y}_i - y_i)^2$ into a sentence of instructions. (Ch. 2)
5. Name the four stages of the training loop in order, and say what each does. (Ch. 3)
6. Explain why gradient descent is described as “purely local”. (Ch. 3, 11)
7. Write down the linear regression model and explain what a single weight means in words. (Ch. 4)
8. Explain why squaring the error changes the model’s behaviour, not just the scale. (Ch. 4)
9. Give one reason the closed-form solution for linear regression is not used in practice. (Ch. 4)
10. Explain what the sigmoid does, in one sentence. (Ch. 5)
11. Explain why logistic regression is a **linear** classifier despite the S-curve. (Ch. 5)
12. Explain why cross-entropy punishes confident wrong answers more heavily than squared error would. (Ch. 5)
13. Compute the Gini impurity of a node containing 8 of class A and 2 of class B. (Ch. 6)
14. Explain what “greedy” means in tree building, and one consequence. (Ch. 6)
15. Give two advantages and two limitations of decision trees. (Ch. 6)
16. Explain what a random forest does differently, naming both sources of randomness. (Ch. 7)
17. Explain why a forest can memorise its training data and still generalise well. (Ch. 7)
18. Define the margin and explain why maximising it is desirable. (Ch. 8)
19. Explain what a support vector is, and what follows from only they mattering. (Ch. 8)
20. Explain the kernel trick in two sentences **without using the word kernel**. (Ch. 8)
21. State what the hyperparameter C controls in a soft-margin SVM. (Ch. 8)
22. Define underfitting and overfitting, giving the train/test signature of each. (Ch. 9)
23. State the bias-variance decomposition and explain all three terms. (Ch. 9)
24. Name four remedies for overfitting and say which one increases bias. (Ch. 9)
25. Write out the XOR truth table and explain why no perceptron can compute it. (Ch. 10)
26. State what the perceptron convergence theorem guarantees **and what it assumes**. (Ch. 10)
27. Explain what took seventeen years between 1969 and 1986. It was not the idea of a hidden layer. (Ch. 10, 14)
28. Write down the gradient descent update rule and explain the minus sign. (Ch. 11)
29. Compare batch, stochastic and mini-batch descent, and say why mini-batch won. (Ch. 11)
30. Name the four learning-rate failure modes and their signatures. (Ch. 12)
31. For $f(x) = x^2/2$, **derive** the range of learning rates for which descent converges. (Ch. 12)
32. Explain how two runs can have identical final losses and opposite problems. (Ch. 12)
33. Explain why local minima are rarer in a hundred dimensions than in two. (Ch. 13)
34. Distinguish a saddle point from a local minimum, and say why the distinction changes what you would do. (Ch. 13)
35. Write the momentum update and explain the velocity term. State one cost. (Ch. 13)
36. State the credit assignment problem in one sentence. (Ch. 14)
37. Explain what backpropagation computes, and what it does **not** do. (Ch. 14)
38. Explain why computing gradients right-to-left is efficient. (Ch. 14)
39. Explain why the forward pass must store intermediate values, and give one consequence. (Ch. 14)
40. Explain what a hidden layer buys you, using the XOR hidden activations as evidence. (Ch. 14)

41. Explain in one line why stacking linear layers without activations is pointless. (Ch. 15)
42. Explain the vanishing gradient problem, including **why** multiplication causes it, with a number. (Ch. 15)
43. Say why ReLU helps, and name its own failure mode. (Ch. 15)
44. Describe what `loss.backward()` replaced in your numpy code. (Ch. 16)
45. Explain what `optimiser.zero_grad()` does and what happens if you omit it. (Ch. 16)
46. Name the three ingredients missing between 1986 and 2012. (Ch. 17)

21.2 Applied: read the numbers

1. A model scores 1.000 on training data and 0.857 on test data. Name the problem, give two remedies, and say why a random forest with the same two numbers might be perfectly fine. (Ch. 6, 7, 9)
2. Two runs of gradient descent both end at loss 1.589. One used $\eta = 0.02$, the other $\eta = 1.98$. Describe what each was doing, and say what you would change in each case. (Ch. 12)
3. A network's loss becomes nan after eleven steps. Give the most likely cause and the first thing you would change. (Ch. 12)
4. A deep sigmoid network trains its last two layers and leaves the first six near their initial values. Name the problem, explain the mechanism with a number, and give the standard fix. (Ch. 15)
5. Your custom loss trains to a plateau at 40% accuracy. You have not run a gradient check. What do you do first, and why? (Ch. 14)
6. You are given the lab's seven-model table. Choose a model for a decision that affects people, and defend it against the objection "but the forest scores higher". (Ch. 4, 7, 9; Week 2 Ch. 12)
7. A colleague reports 94% accuracy. What are the two questions you ask before responding? (Ch. 9)

21.3 Discussion prompts

1. Week 2 argued that a loss function is a values statement. Take one loss from this week and say precisely what it values and what it ignores.
2. The lab's best model is an SVM from the 1990s. What does that suggest about the relationship between a technique's age and its usefulness, and what would you have to believe for the opposite to be true?
3. Backpropagation is the chain rule, which is two centuries old. What does the seventeen-year delay tell you about how progress in this field actually happens, and where would you look for the next such delay?
4. Momentum reaches the minimum's neighbourhood six times faster and ends worse than plain descent. Argue for and against using it by default, then commit.
5. Chapter 17 claims AlexNet contained almost no new mathematics. Is that a criticism, a compliment, or neither?

22 Further Reading

Optional. Nothing here is examinable.

Under an hour. The scikit-learn User Guide’s sections on supervised learning and model selection are unusually well written and are the fastest route to competence with Chapters 4 to 9. 3Blue1Brown’s neural network series, particularly the two videos on backpropagation, is the best visual explanation of Chapter 14 in existence and worth watching even if you found the chapter easy. Karpathy’s “A Recipe for Training Neural Networks” is a short, blunt list of the mistakes in Chapters 12 and 16, written by somebody who has made all of them.

An evening. Karpathy’s **micrograd** is a complete automatic differentiation engine in about a hundred lines of Python; reading it end to end is the fastest way to stop finding `loss.backward()` mysterious. Rumelhart, Hinton and Williams (1986), the paper that popularised backpropagation, is short and surprisingly readable.

A weekend or more. Goodfellow, Bengio and Courville, **Deep Learning**, Chapters 4 to 8, remains the standard reference for everything in this booklet and is free online. Hastie, Tibshirani and Friedman, **The Elements of Statistical Learning** is the definitive account of Chapters 4 to 9, also free online, and considerably more demanding. Bishop, **Pattern Recognition and Machine Learning** is the classic alternative with a Bayesian slant. For the caveat in Chapter 9, Belkin and colleagues (2019) is the paper that named double descent.

Appendix: How Week 3 Maps to the Module

Module element	Week 3 connection
LO2 (characterise the training of model families)	Chapters 3 to 15; assessed in the Week 6 quiz.
LO4 (tools and frameworks)	Chapter 16, PyTorch; assessed in the Week 6 quiz.
Indicative content 2, 3, 7	Classical ML and early neural networks; backpropagation and GPU acceleration; PyTorch as the working framework.
Week 1 (history)	Collected here: the XOR wall and its escape route (Ch. 10, 14); the three ingredients of 2012, from inside the loop (Ch. 17); LeNet in production in 1989.
Week 2 (responsible AI)	The loss function as a values statement (Ch. 1, 4, 5); interpretable-by-design models as a live choice rather than a sacrifice (Ch. 4, 6); the reweighting lever of Week 2's Ch. 15 is a loss modification; the hyperparameter you did not choose is Week 2's threshold argument in a new place.
Week 4 (model families)	“Choosing a model is choosing an assumption” becomes “connectivity is an assumption about the data”. The boundary shapes of Chapters 4 to 8 are the low-dimensional version, and Ch. 14's change of coordinates is the mechanism.
Week 5 (science-grounded)	A hidden layer learns a change of coordinates; a physics-informed network is told one. Chapter 14's closing idea is Week 5's premise, and Ch. 17 explains why it matters economically.
Weeks 7 to 9 (LLMs, scale-free, lean)	The same loop at a different scale. The forward-pass memory cost of Ch. 14 is why training and inference are budgeted separately, and the hardware argument of Ch. 11 is why some architectures win.
Week 12 (project)	The seven questions of Chapter 18 are the evaluation section of your critical brief.
Gradient Descent Explorer	Chapters 11 to 13 as a manipulable object: move the learning rate and watch all four failure modes happen to you.

End of the Week 3 Reference Booklet. Next: Week 4, model families, where the first box of the loop stops being an afterthought and becomes the whole design.